

Engineering Efficient Self-Play Chess: Search, Replay, and Throughput Under Limited Compute

Bertil Braun

contact@bertil-braun.de

Abstract

How strong can an AlphaZero-style chess system become under limited training compute when its entire learning loop is engineered for efficiency? We train from random initialization through searched self-play on a single eight-GPU node for 2.5 days. The resulting 6.32-million-parameter model reaches 3,251 benchmark Elo [3,206, 3,297] at 100,000 searches per move (roughly five seconds of thinking time) against a fixed-node Stockfish 13 ladder. The run ingests 3.25 million completed games, involves roughly 195 billion search simulations, and makes 836.6 million training presentations. We investigate search allocation, replay and restart-state selection, policy representation, progressive model sizing, quantized inference, and throughput engineering. Alongside the retained design, we document plausible alternatives that failed to improve the complete learning loop or did not justify their cost. The reported strength is a result of the integrated system, not an isolated Elo gain attributable to any single choice.

Keywords: AlphaZero, chess, self-play, Monte Carlo tree search, replay, GPU inference, quantization, model growth, fixed-node evaluation

1 How far can efficient self-play go?

An AlphaZero-style chess player [1] improves by searching its own games, learning from the resulting positions, and repeating that cycle with a stronger network. Search creates targets; replay decides which targets persist; training absorbs them; and evaluation must distinguish progress from noise. Under a limited budget, improving any one step matters only if the complete loop produces stronger play within the available time.

This study asks how strong that loop can become on a single eight-GPU node when the system is engineered for efficiency. Over **2.5 days** of training from random weights on searched self-play games, the run produced a 6.32-million-parameter model. It reached **1,658 benchmark Elo without search** and **3,251 benchmark Elo at 100,000 searches per move**—roughly five seconds of thinking time—against the project’s fixed-node Stockfish 13 ladder. Table 1 gives the match results; Chapter 8 follows the model’s improvement across search budgets.

The result depends on the whole learning loop. The network’s representation affects both what it can learn and how quickly it can supply search. Native search, batched inference, and TensorRT make enough searched games available for training; replay and training must then turn those games into stronger play. Several plausible optimizations improved a local metric without improving that complete loop.

1.1 Scope and contributions

Chess is the subject of the study. The runtime also supports Go, and KataGo’s fast and full searches inspired one of the approaches tested here [7]. We briefly considered 7×7 and 9×9 Go as cheaper settings for tuning chess hyperparameters. In those exploratory games, we observed a strong first-player advantage, short trajectories, and a value target that learned quickly. Useful Go tuning appeared unlikely to transfer directly to chess, so we kept chess as the focus.

We examine the choices that shaped the final recipe: search allocation, graph search and caching, policy representation,

model sizing, replay and restart states, resignation, auxiliary targets, and quantized inference. The report also traces the throughput needed to supply searched games and studies three failures with transferable lessons about self-play targets, deployment fidelity, and model promotion.

A live chess demonstration is also available [12].

1.2 Roadmap

Chapter 2 presents the chess evaluation. Chapter 3 explains the AlphaZero learning principle and the system that implements it. Chapter 4 examines search, replay, and network design, followed by throughput engineering in Chapter 5. The three failure studies in Chapter 6 lead into training progress in Chapter 7 and playing strength in Chapter 8. Appendix D specifies the final recipe.

2 Evaluating the chess system

We measure thinking effort in search visits rather than seconds. For a given model and search configuration, this budget is hardware-independent: faster hardware finishes sooner instead of receiving more search. Fixed budgets also keep background load from changing the amount of search performed. We report approximate thinking time separately to give these budgets a practical scale.

The most direct test of the system is whether its final model wins games. We played it against Stockfish 13 at fixed search limits, starting from 50 openings and playing each once with each colour. The model played without search and at four progressively larger search budgets. For each budget, we tested two Stockfish limits rather than trusting a single opponent.

2.1 Final playing strength

Table 1 gives all ten matches. Each row contains 100 games; wins, draws, and losses are from our model’s perspective. The bold rating for each model budget comes from the opponent against which it scored closest to 50%, where the rating estimate needs the least extrapolation.

Table 1: Final checkpoint against fixed-node Stockfish 13

Model searches	Parallel	Stockfish nodes	W/D/L	Score	Benchmark Elo (95% CI)
Policy only	–	1,000	32/24/44	0.440	1,658 [1,608, 1,710]
Policy only	–	2,000	16/22/62	0.270	1,717 [1,638, 1,790]
100	1	5,000	51/28/21	0.650	2,328 [2,276, 2,384]
100	1	10,000	39/18/43	0.480	2,456 [2,400, 2,512]
1,000	1	20,000	47/40/13	0.670	2,823 [2,774, 2,873]
1,000	1	50,000	21/48/31	0.450	2,925 [2,875, 2,977]
10,000	4	50,000	45/40/15	0.650	3,068 [3,023, 3,120]
10,000	4	100,000	30/44/26	0.520	3,114 [3,065, 3,163]
100,000	16	100,000	51/38/11	0.700	3,247 [3,192, 3,305]
100,000	16	200,000	25/56/19	0.530	3,251 [3,206, 3,297]

At the deepest budget, the model scored 3,251 benchmark Elo against the harder opponent and 3,247 against the other: the two measurements agree closely. The ratings use a published calibration of Stockfish’s fixed node limits [9]. Chapter 8 examines how strength changes with search; Appendix B gives the rating calculation and interval method.

2.2 Reading the component experiments

The rest of the paper asks why the system reached that result. A faster inference engine can supply more search, but that only helps training if the system finishes more games and admits useful positions to replay. A lower loss on stored positions can indicate a better fit, but the real test is whether the next model plays better. We follow each proposed improvement as far through this chain as the experiment measured.

For online comparisons, we matched starting weights, replay, evaluation, and hardware where possible. Smaller frozen-data tests helped screen ideas before expensive self-play runs. Because the final recipe combines many changes, we attribute a separate strength gain to a component only when a comparison actually measured one.

3 Background and system design

This chapter introduces the AlphaZero learning loop and the system used to run it under limited compute. It follows the path from network predictions through search and self-play to replay and training, establishing the foundations for the experiments in Chapter 4.

3.1 Learning through search

AlphaZero learns a chess player without examples of human play [1]. Its network has two jobs: the *policy* assigns probabilities to moves, while the *value* estimates the outcome of a position. Monte Carlo tree search (MCTS) brings these predictions together. It explores moves using both their policy probabilities and the results found so far, balancing promising continuations against less-explored alternatives. At a new leaf, the network evaluates the position; that value is backed up along the path to inform subsequent exploration.

Search can therefore challenge the network’s first impression. A promising move may reveal a strong reply for the opponent, while a less obvious move may lead to better positions. The root visit distribution becomes a policy target, and the completed game’s outcome teaches the value prediction. Training folds this experience back into the network. Better predictions then guide later searches towards more useful con-

tinuations, sustaining a cycle of search, self-play, and learning.

This is the central opportunity under limited compute: spend search to discover improvements, then learn enough from those improvements that the next search starts from a stronger player. Cheap searches that mostly repeat the network’s initial preference may produce many positions but little new policy information. Very deep searches can make good targets too expensive to supply in sufficient quantity. The system must make both searching and learning affordable.

3.2 Related work

AlphaZero establishes the self-play learning framework [1]; KataGo shows how substantially its compute requirements can be reduced through changes to search, training, and network architecture [2]. KataGo is the closest practical precedent for this study’s efficiency focus. Its fast/full search schedule, auxiliary objectives, and self-play methods [7] motivated several investigations here. Their usefulness still depends on the game: completing more long Go games and supplying more searched chess positions need not favour the same allocation of compute.

Several narrower lines of work address where that compute should go. Dynamic simulation MCTS studies when to stop search [3], and targeted search control starts self-play from archived states to explore beyond ordinary opening trajectories [5]. Prioritized experience replay changes which stored examples are learned from again [4], while Monte Carlo graph search shares work across paths reaching the same state [6]. These ideas motivate the allocation, replay, restart, and reuse experiments. This chapter explains the resulting system; Chapter 4 examines how the alternatives performed, and Chapter 5 measures the throughput needed to run it.

3.3 One learning cycle

The learning loop has three sources of work: actors play games, the replay pipeline stores their experience, and the trainer updates the network. An *actor* is a self-play process that advances many games concurrently rather than waiting for each game to finish before starting another. All those games use a published set of network weights to guide their searches.

Figure 1 follows the flow between these components. Python orchestration starts and coordinates the work. Within each actor, native C++ search requests policy and value predictions for newly explored positions. Requests from different

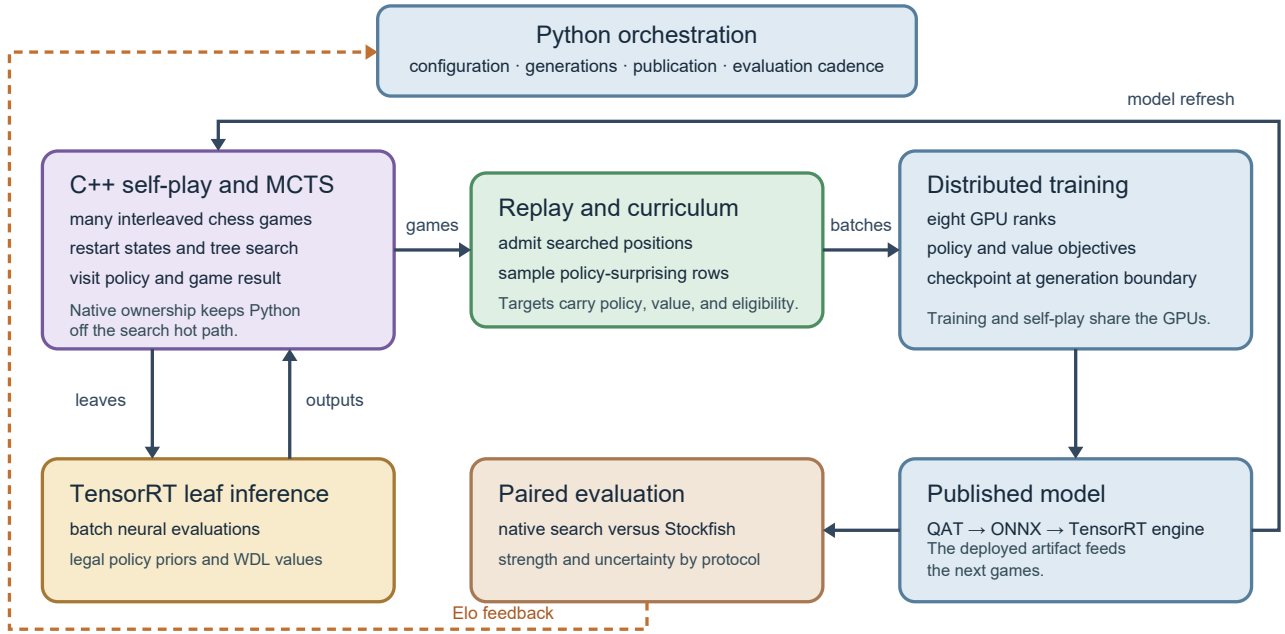


Figure 1: Self-play turns a published network into searched games, replay supplies their positions to the trainer, and updated weights return to the actors. Separate matches measure the player’s progress.

games are grouped into batches and evaluated on the GPU by TensorRT, the optimized inference engine. While a batch is being evaluated, other ready games can advance. Returned predictions let the waiting searches update their trees and eventually choose their moves.

A finished game supplies a sequence of positions, the search policy at each recorded move, and its final outcome. The replay pipeline converts that sequence into training examples. The trainer draws batches from the accumulated examples and adjusts the weights to better predict their policies and outcomes. After a block of optimizer steps, publication prepares the updated network for the actors. The next games then benefit from what the learner has absorbed.

Some actors keep playing while training runs, so the system can produce the next games while learning from earlier ones. Evaluation runs alongside this loop using the native chess engine to play matches against Stockfish. These matches measure progress; they do not become self-play training data.

3.4 Why the search loop stays in C++

The language boundary follows the frequency of the work. One played move requires hundreds or thousands of tree traversals. Each traversal may update a board, generate legal moves, select a continuation, encode a position, and back up a value. If Python dispatches these steps individually, interpreter and boundary-crossing costs recur throughout the most frequently executed part of the system. Even a fast GPU can then spend time waiting for the CPU to prepare its next batch.

C++ owns the complete inner loop: game state, search trees, traversal, and the inference request pipeline. An actor can retain its trees and buffers, interleave independent games, and submit batches without returning to Python for each search leaf. After a move is played, the relevant subtree becomes the next root, preserving useful work rather than rebuilding the search from nothing.

Python operates at a coarser scale. It coordinates processes, manages replay and publication, and trains the model through PyTorch and its distributed libraries. These tasks benefit from the existing numerical ecosystem and ease of experimentation without placing Python on the path of every simulation. The purpose of the split is not to make every operation native: it is to keep the expensive repeated work fast enough to supply the learner with games.

3.5 Chess representation and outputs

Figure 2 shows the network from its $52 \times 8 \times 8$ chess input through a shared residual backbone to its output heads. The input planes encode pieces, recent moves, and game-state features from the side-to-move perspective. Castling rights, repetition, and the halfmove clock distinguish positions whose boards look identical but whose legal continuations or draw conditions differ. Appendix D, Table D1 gives the complete channel-by-channel specification.

The backbone learns a common spatial representation for move selection and position evaluation. Its 14 residual blocks, each with 160 channels, refine board features by adding learned corrections to the preceding representation. Every second block also pools features across the board and feeds the resulting global context back into the spatial features. This lets a local feature respond to the wider position without relying solely on successive local convolutions to propagate that information.

A *head* maps these shared features to a particular prediction. The policy head represents each square with learned origin and destination vectors; their dot products score moves, with additional offsets distinguishing promotion pieces. The fixed encoding contains 1,880 actions: 1,792 origin–destination pairs along sliding-piece rays or knight jumps, plus 88 promotion moves distinguishing the four promotion pieces. King and ordinary pawn moves use the same ray pairs. Mapping the scores to these actions, masking illegal moves, and nor-

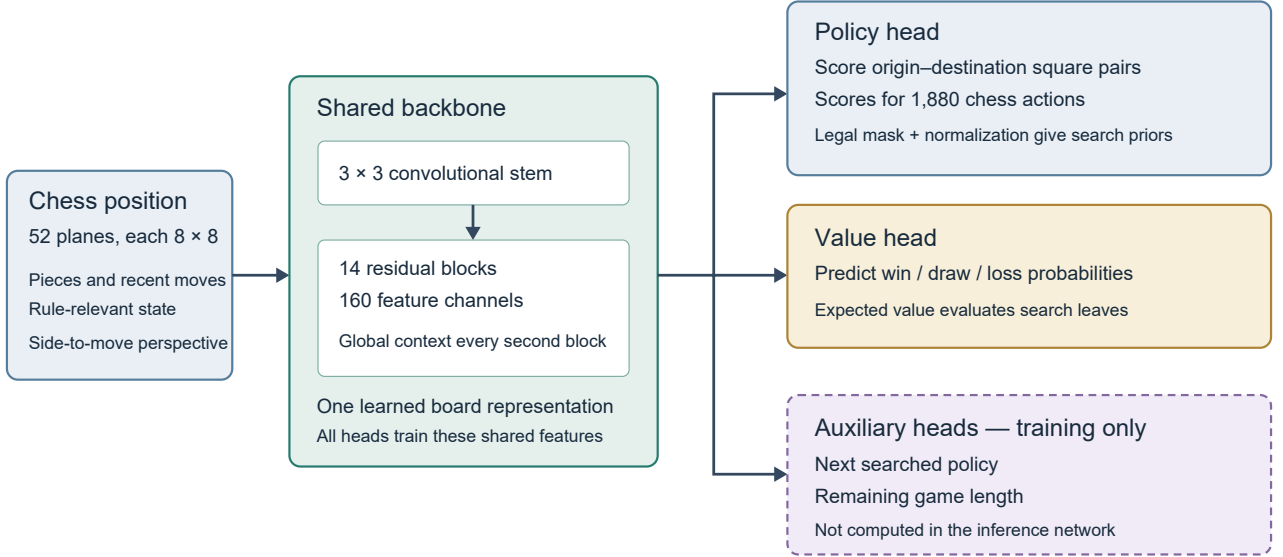


Figure 2: The final chess network. A shared 14-block, 160-channel backbone processes 52 input planes. Policy and value heads provide the predictions used by search; the two auxiliary heads contribute only during training.

malizing produces the move probabilities that guide search. The value head instead predicts win, draw, and loss (WDL). Search uses the win probability minus the loss probability as its scalar estimate; training retains the full distribution, distinguishing a likely draw from equally likely winning and losing outcomes.

Both objectives train the same backbone, sharing the cost of extracting board features. Two auxiliary heads add supervision for the next searched policy and remaining game length. Their predictions are not needed to choose moves, so they are omitted from the inference copy while the backbone retains the features learned from those tasks. Appendix D specifies the output shapes and action encoding.

3.6 Search and self-play

At each move, search builds a tree rooted at the current position. Each traversal follows the PUCT selection rule, balancing the value found for a continuation against the benefit of investigating it further:

$$a^* = \arg \max_a [Q(s, a) + U(s, a)],$$

$$U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{\max(1, N(s))}}{1 + N(s, a)}. \quad (1)$$

Here, $P(s, a)$ is the policy prior for move a in position s , $Q(s, a)$ its estimated value for the player to move, and $N(s, a)$ its visit count; $N(s)$ counts visits to the parent position. The exploration constant is 1.5. The first term rewards continuations that search already considers strong. The second favours moves with a promising prior but relatively few visits, decaying as they receive attention. Unvisited moves start from a reduced parent-value estimate until a traversal supplies their own evidence.

Reaching a new leaf triggers a network evaluation, unless the game has ended and its result is already known. Backing up that value updates the visits and accumulated estimates along the selected path, reversing perspective at each ply. Successive traversals therefore test the network’s prefer-

ences against increasingly explored replies, rather than merely resampling its initial move probabilities.

For example, the network may initially prefer a move that wins a pawn. Search can discover that the opponent then has a dangerous reply, lower its estimate of that continuation, and spend more visits on a safer alternative. The training signal is not just which move was finally played: the distribution of root visits records the search’s relative preference across the available moves. That distribution becomes the policy target.

Self-play must explore as well as exploit its current knowledge. Noise added to the root prior makes games consider different continuations, and a temperature setting controls how sharply played moves follow the visit leader. The search budget rises in stages from 300 to 800 visits as training progresses. Useful visits from the retained subtree contribute to the next move, while each new search refines the choice from its new root.

Many independent games share inference batches. This uses the GPU efficiently without requiring every tree to search multiple leaves at once. When per-tree parallelism is used, several traversals can be waiting for results simultaneously; temporary reservations discourage them from all choosing the same path. They still make their choices without the other pending results, creating the strength-versus-latency tradeoff examined later.

Games need not all repeat the ordinary initial position. Roughly half start after a short random legal opening, and the other half use archived self-play positions with interesting alternatives left to explore. A restart plays a different branch from such a position, creating new experience rather than simply replaying its old target. Ordinary starts maintain whole-game coverage while restarts spend some of the budget on unresolved choices.

Games normally supply win, draw, or loss labels when they end. Resignation avoids spending search on clearly lost positions, but some games are forced to continue so the system can estimate how often resignation would be wrong. A maximum game length also prevents extremely long games from con-

suming unlimited compute. At that limit, one final full search estimates the unfinished position’s value instead of assigning it an arbitrary result.

3.7 Replay and materialization

A *replay buffer* separates the order in which experience is generated from the order in which it is learned. Adjacent positions in a game are highly correlated; mixing examples from many games gives each optimizer batch a broader range of openings, middlegames, and endings. Reuse also amortizes the cost of search: a useful position can contribute to several updates without requiring another game to generate it again.

Each example stores the encoded position, its legal actions, the searched policy, and an outcome target. For a finished game, the outcome is expressed from the side to move at each stored position. For a capped game, the final searched estimate supplies a substitute target. Random opening moves and reconstructed restart prefixes have no search policy of their own, so they are not treated as searched training examples.

Materialization is the conversion from complete game trajectories to these rows. The rows live in a circular memory-mapped store: as the active window fills, new examples replace its oldest contents, and trainers read batches without loading the whole buffer into each process. Search policies are stored sparsely, retaining the actions with visit mass rather than writing a full mostly empty vector for every position.

Replay capacity grows from 600,000 towards 20 million positions. The small early window lets new experience quickly replace the weakest early play; the larger later window preserves more variety. Sampling combines a uniform component with a preference for positions where search changed the network’s move probabilities substantially. Such positions offer a plausible learning opportunity, while uniform sampling prevents the learner from seeing only unusual or difficult cases. Each selected row has the same loss weight; priority changes how often it is seen.

Data supply also sets the pace of training. The chosen reuse ratio permits four training presentations per new replay position. A 500-step block with a batch of 2,048 therefore requires 256,000 new positions to support its 1,024,000 presentations. If the actors have not supplied enough, training waits. This keeps a faster optimizer from merely making more passes over an unchanged pool of experience.

3.8 Training

Training distills search and game outcomes into network predictions. For one position, the objective is:

$$\begin{aligned}\mathcal{L} &= \text{CE}(\pi, p) + \text{CE}(z, v) + \mathcal{L}_{\text{aux}} \\ &= - \sum_a \pi_a \log p_a - \sum_{k \in \{W, D, L\}} z_k \log v_k + \mathcal{L}_{\text{aux}}.\end{aligned}\quad (2)$$

The searched policy π and outcome target z supervise the predicted move probabilities p and WDL probabilities v . Both primary losses have unit weight; the auxiliary term combines next-policy and remaining-length losses with weights 0.15 and 0.1. Their gradients meet in the shared backbone, so a feature useful for predicting outcomes can also improve the representation from which move preferences are learned.

Outcome discounting softens targets for positions far from the end of a game. It was introduced, alongside search dis-

counting, to favour earlier conversion of winning positions; Section 4.2 discusses the motivation and unresolved benefit. A small contribution from the position’s searched value is also blended into the training target. The auxiliary losses add next-policy and remaining-length supervision when those labels exist. An unfinished game cannot reveal its true remaining length, so that auxiliary loss is omitted rather than trained towards a fabricated zero.

Training uses distributed data parallel across all eight GPUs with a global batch size of 2,048.

Updates are grouped into blocks of 500 optimizer steps. A block followed by publication is a *generation*, and a saved set of model weights is a *checkpoint*. Half the self-play actors continue working while a block runs. This overlap is valuable because game production and training use the shared hardware differently, but also means that saving search work will shorten the full cycle only when search was delaying the next block.

3.9 Progressive models and publication

Starting with the final model size would spend its full inference cost even while the player is still learning basic chess. Progressive sizing starts with a smaller, faster network. Its searches are cheaper, supplying more games during this early stage. As improvement slows and model capacity becomes more limiting, a larger network can make better use of the experience already collected.

The ordinary transition trains a larger candidate on the same replay alongside the active model. Since the candidate starts independently, it first needs catch-up training before it can replace an already competent player. Paired head-to-head matches test whether it has caught up. These matches decide the occasional change in model size; they are not a gate applied to each routine training update. Within an active size, updated weights are published after training blocks without requiring them to defeat the previous generation.

A second approach initializes the larger network to reproduce the smaller network’s current predictions. This function-preserving growth avoids much of the initial catch-up, but may also keep learning close to the smaller model’s internal representation. The final reported player uses the successful small-to-medium path; the limited larger-model continuation did not establish a further strength gain.

Publication bridges training and inference. Training retains optimizer state and auxiliary heads, while search needs only a fast policy-and-value predictor. A separate inference copy removes those unused heads and combines operations where possible. It is exported through ONNX, a model representation that TensorRT can compile into an optimized GPU engine. Refitting replaces weights in a prepared engine rather than rebuilding it after every training block.

Most backbone convolutions use eight-bit integer arithmetic to accelerate inference. Quantization-aware training simulates the rounding and clipping this introduces, letting the network adapt before deployment. The exported copy is checked against the training model on representative positions so the speedup does not silently change its move preferences or value predictions. Once prepared, it becomes the model used by subsequent self-play.

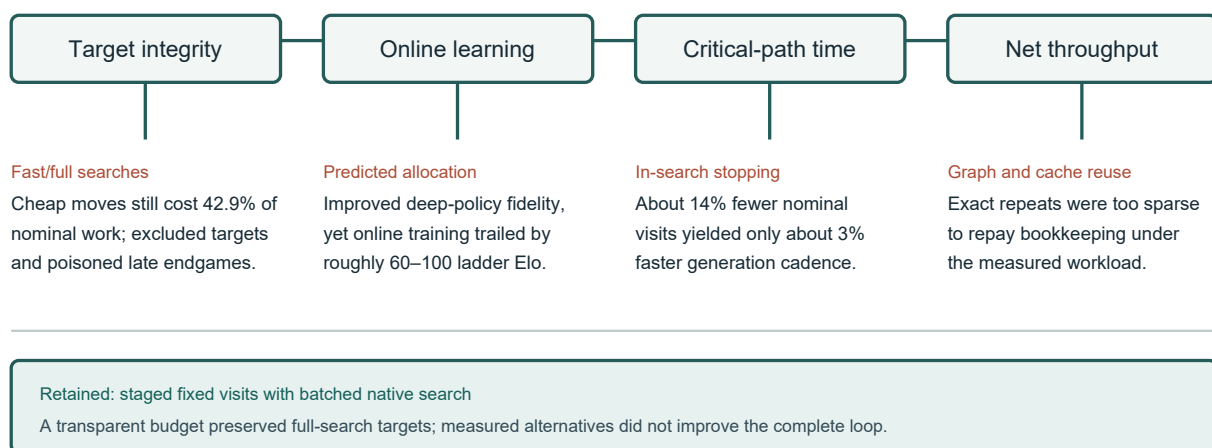


Figure 3: Why the search alternatives were not retained. Fast/full search discarded useful targets; predicted allocation improved policy agreement but weakened learning; stopping saved search without much faster training; and exact graph or cache reuse saved too little work to repay its overhead.

3.10 Evaluation

Training loss shows how well a network fits replay, but fitting those examples is not the same as playing stronger chess. Evaluation therefore plays separate matches against fixed-node Stockfish opponents. Each opening is played twice with colours reversed, reducing the influence of which side received the easier starting position. Wins, draws, and losses determine the match score and its implied rating on the project’s calibrated ladder.

During training, short 64-search and policy-only evaluations track progress at modest cost. They answer different questions: policy-only play tests the network’s immediate move choice, while searched play tests how useful its policy and value are together in a tree. Larger final matches repeat the assessment across increasing search budgets, as reported in Table 1.

The resulting feedback completes the experimental loop. A change that improves a local loss or throughput measure still has to help produce a stronger player within the available training time. The following chapters use this system description to examine those choices, rather than treating local speed or fit as an end in itself.

4 Research investigations

This chapter examines the approaches we tested, their results, and the choices retained in the chess system. Section 4.1 concerns how much search to spend and where; Section 4.2 examines self-play data, restart states, and replay; Section 4.3 considers network architecture and training. Together, these investigations connect the cost of generating experience to how effectively the network learns from it.

4.1 Spending search where it matters

More search makes an individual move stronger. With the same frozen network and opponent, raising the budget from 200 to 1,600 visits raised its match score from 0.318 to 0.748. Training poses a different question: could some moves use fewer visits so that self-play finishes more games without teaching the next model worse policies? Cheap moves might

save computation, but the saved work matters only if it increases useful training data or shortens the learning cycle.

We tried fast/full searches, several ways to allocate or stop search according to position, and reuse through a graph or inference cache. The retained approach increases a fixed visit cap in stages and applies that cap to every recorded move. The experiments below distinguish four tests that can give different answers: immediate move strength, agreement with a deeper policy, strength of the next trained model, and throughput.

4.1.1 The fixed-budget baseline

The baseline spends a fixed number of visits on each move. Within that budget, the PUCT selection rule combines the value found for a move with an exploration bonus based on its policy probability and visit count. A promising but little-explored move receives more attention; a heavily explored move must justify further visits through its value. New leaf evaluations update the choices along their search paths, with the perspective alternating between the two players. Batching these evaluations across games makes the search practical on a GPU.

Against the same opponent, score rose from 0.318 at 200 visits to 0.537, 0.580, 0.662, and 0.748 at 400, 600, 1,000, and 1,600 visits. Across the better-resolved middle and deep part of the sweep, the fitted trend was about 81 Elo per doubling. Individual adjacent comparisons were noisy, so the overall trend is more credible than any single increment. Search was still changing its policy target too: at 600 visits, only 72.4% of positions selected the same best move as the network’s own 10,000-visit reference. These frozen-network measurements establish the value of search depth, not an optimal training schedule.

The training cap increases through 300, 400, 500, 600, and finally 800 visits. Cheap early search supplies games quickly; deeper later search gives the improving network richer targets. Retaining the relevant subtree after a played move also saves work: some visits at the new root have already been performed during the previous move.

4.1.2 Why fast and full searches did not transfer

A scheme inspired by KataGo’s self-play methods [7] used cheap searches to advance most moves and full searches on a random minority, which alone became primary policy targets. The trade is attractive in 19×19 Go: long games make finished outcomes expensive, so cheap moves can supply more independent value targets. Chess games are shorter, their outcomes and positions are often clearer, and the value objective was already learning. The transferred recipe thus spent search on moves whose positions were discarded as primary training rows without showing that the extra completed games compensated for the lost policy-target density.

With a quarter of moves searched to 600 visits and the rest to 150, cheap moves consumed 42.9% of nominal search work despite being excluded as primary targets. They still selected played moves, affected terminal outcomes and retained trees, could supervise a preceding row’s next-policy target, and influenced which positions entered the restart archive.

The mixed workload also interacted badly with batching. Once the cheap searches finished, only the full-search minority remained. In a 512-game test this tail left about 128 active trees and filled only 86 of 320 available batch slots with one leaf per tree. Allowing four leaves per tree raised the average batch to about 268 and improved search throughput by roughly 20%, but it did so by making each search less serial. A policy intended to save compute thus created pressure to accept a search-quality tradeoff merely to keep the accelerator occupied.

The retained design searches every recorded move to the scheduled cap, preserving both policy targets and endgame coverage.

4.1.3 Three attempts to allocate search adaptively

The first attempt asked whether search could stop when the most-visited move looked uncatchable. An audit of completed trees found apparent room: an optimistic reconstruction suggested that a rule requiring at least 75% of the cheap budget might remove 15.3% of cheap-search visits, or 6.38% of all nominal limits if full searches saved nothing. But the records contained final visit distributions, not the intermediate traces needed to identify when the leader first became safe. They also omitted enough state to reconstruct retained visits and forced-playout-pruned targets. The reconstruction could suggest where to look for savings, but it could not tell a running search when to stop.

A wider offline study found a deeper problem. Concentrated visits did not consistently mean that search was finished: very diffuse and already-decided positions gained little, while moderately concentrated but contested positions gained most. Simple concentration thresholds tended to stop in the very region where more search was useful. The rule-based approach was therefore declined before an online strength match.

The next allocator replaced the rule with a learned prediction, related in aim but not identical to dynamic simulation stopping [3]. An auxiliary head estimated, for several candidate budgets, how far that budget’s policy would differ from a deep-search policy. A correction based on the current root statistics refined this prediction, and a controller adjusted allocation to keep average spend near its target. At approximately matched mean spend it captured about 23% of the possible

reduction in policy divergence; its mean target fidelity resembled roughly 1.18 times uniform search at 0.967 times the spend. It also behaved plausibly, assigning more work to contested positions.

Learning nevertheless deteriorated. Repeated online attempts trailed comparable non-adaptive training by roughly 60–100 ladder Elo. More than a third of positions received an average budget fraction near 0.36, and almost 9% received one eighth of baseline search. Those shallow policies were close to the network’s own prior yet were trained at full weight. One explanation is that closeness to a deep policy does not measure how much a target will improve the next network. Easy-to-predict targets can agree with deeper search while teaching little beyond what the network already knows. The controller improved policy agreement, but its online tests produced worse learning.

The final adaptive system moved the decision inside search, where a learned stopper could observe the evolving tree rather than predict difficulty in advance. Its decisive test started from the same checkpoint, optimizer, and rebuilt replay state for every arm. The most aggressive setting skipped about 14% of nominal search, and its internal credit-wait measurements changed in the expected direction. Yet generation cadence improved by only about 3%. Self-play overlapped the optimizer, so most of the removed search was slack rather than critical-path work. The paired strength differences, calculated as baseline minus stopper, were $+1.7 \pm 9.9$ Elo and -4.2 ± 10.1 Elo (standard errors) for the two settings: neither resolved a strength effect. At the observed learning rate, the cadence gain was worth only about one Elo over three hours, below the experiment’s resolution.

The three approaches failed for different reasons. The threshold rule lacked an identifiable safe signal; the predicted allocator improved policy fidelity but not learning; and the in-search stopper removed mostly off-critical-path work while search and training shared the available GPU capacity.

4.1.4 Parallel leaves: buying latency with search quality

Batching across many independent games is the cleanest way to feed the accelerator, but the number of active roots eventually runs out. The engine can then keep several leaf traversals in flight from one root. Virtual reservations discourage those traversals from selecting the same path, yet every selection is based on a tree that is missing the other in-flight results. Parallel search is therefore not serial MCTS executed faster. It exchanges fresher decisions for larger batches and lower latency.

To measure this cost, the batch must have room for multiple leaves from each tree. Otherwise, many independent games can fill it before per-tree parallelism has any effect. A sweep that deliberately provided this room estimated a loss of 6.4 ± 4.7 Elo per doubling of parallel leaves. With that uncertainty, comparisons at specific search budgets are more informative than treating the estimate as a general rule.

The tradeoff also depended on the total budget. Table 2 pairs strength with match duration in a fixed-network sweep at 1,000 searches against 20,000-node Stockfish.

Four- and sixteen-way parallelism reduced match time by roughly factors of five and eleven. Their strength differences were smaller than the overlapping match intervals. At only

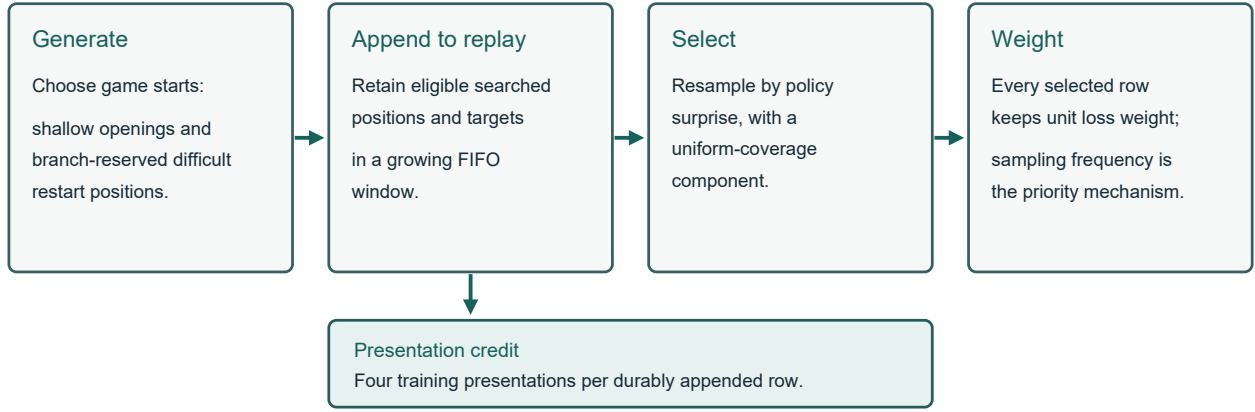


Figure 4: Starting positions shape the games that are played. Their searched positions enter replay, where sampling determines which examples the learner revisits. New examples also set the pace of optimizer updates.

Table 2: Parallel search at 1,000 visits against 20,000-node Stockfish

Leaves	Elo	Time (min)	Elo change
1	2,820	18.1–18.5	0
4	2,800	3.4–3.8	–19
16	2,780	1.3–1.8	–45

100 searches, however, sixteen-way parallelism reduced the central strength estimate by about 235 Elo. A deeper search will eventually visit more of the temporarily suboptimal leaves selected from stale state, making the same concurrency less disruptive.

The resulting choice differs between training and interactive play. Many independent self-play games can fill batches without much per-tree parallelism. A player waiting for one move has only one root, making parallel leaves more useful. In either case the budget must be large enough to tolerate the less-informed selections.

4.1.5 When a tree became a graph

Chess appears rich in transpositions: different move orders often reach the same board. The project tested whether Monte Carlo graph search, as in Czech, Korus, and Kersting [6], could share neural evaluations, descendants, and search statistics across those paths. Canonical nodes held shared state information while parent/action edges kept local PUCT statistics. Sharing a position in this way is more demanding than looking it up in a table: results must be propagated correctly through its incoming paths, and the graph must remain usable after a played move.

The limiting fact was chess-state identity. Pieces and side to move are not enough: castling rights, en-passant state, the halfmove clock, and repetition-relevant history can change the legal result. Merging positions that differ on those fields would create an approximate algorithm with different game semantics. Under exact equality, most apparent board transpositions disappeared.

At ordinary budgets, useful sharing was negligible. Only 0.0249% and 0.1769% of neural evaluations were avoided at 1,000 and 10,000 searches, while the graph was 8.63%

and 8.28% slower. At 30,000 and 60,000 searches, table hits rose to 2.37% and 3.46%, but avoided evaluations remained approximately 0.0001% and 0.0348%; throughput was still 7.06% and 5.76% lower. Structural counters confirmed that shared descendants and statistics were active. Exact reuse was orders of magnitude too sparse to repay the bookkeeping cost in this implementation, so it was not taken to a final strength match.

4.1.6 Why inference caching found little to reuse

Inference caching asked a narrower question: if an identical encoded neural input reappears under the same model, can its policy and value outputs be reused without sharing search state? Two investigations answered complementary parts of it.

First, a real bounded, sharded cache was shared by the search threads within each self-play process. With eight processes per GPU, three search threads per process, and capacity for 1.5 million entries per process, its hit rate was 0.970%. Disabling the cache made game updates about 0.88% faster in the short stochastic comparison and reduced summed worker peak memory by about 7.12%. The implemented cache consumed memory without demonstrating a speedup.

A later audit measured the upper bound available to a wider cache before building one. An unbounded tracker shared across one search executor observed exact encoded inputs but deliberately evaluated every position. With diverse starts and production-style progression, repeats were 1.33% at 150 searches, about 4.24% at 800, and about 3.5% in the mixed workload; same-batch duplicates were essentially absent. Following retained trees for several moves raised the rate only to roughly 4%. The tracker itself cost about 3.65% throughput and grew without bound. A real cache would additionally pay for output storage, synchronization, eviction, and device transfers while retaining fewer entries.

The wider design was declined before implementation. Together with the implemented per-process cache, its opportunity audit showed little reuse available under this workload.

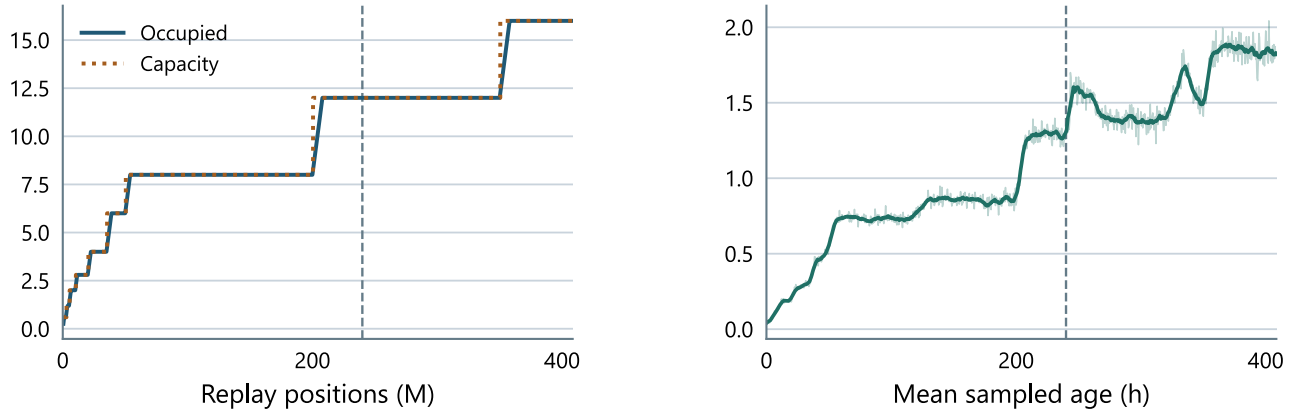


Figure 5: Replay occupancy follows the expanding capacity during final training. The mean age of sampled positions grows to roughly two hours; age is measured from position creation. Both horizontal axes show completed optimizer steps in thousands. The dashed line marks the small-to-medium model transition.

4.1.7 The retained search design

Fixed visits supply a consistent amount of search to each recorded move. Exploration prevents this search from simply repeating its favourite opening: root noise perturbs the move prior, and temperature controls how strongly move selection favours the visit leader. Forced root playouts ensure that alternatives receive some attention; their visits are pruned from the training target when they reflect forced exploration rather than a genuinely preferred move.

For an unvisited move, first-play urgency starts from a reduced parent value rather than an optimistic default. A 0.99 per-ply discount favours nearer favourable outcomes, while retaining 60% of subtree visits after a played move reuses analysis without allowing old statistics to dominate completely. Together with native batching, these mechanisms make fixed-budget search both exploratory and affordable. Appendix D specifies the recipe; Appendix D gives the numerical settings.

4.2 Getting more learning from each game

The data strategy determines both the distribution of self-play experience and its contribution to learning. Restart states direct new games towards unresolved alternatives; replay capacity and sampling determine which searched positions remain available and how often they are trained on. Resignation and cutoff handling affect the reliability of their outcome targets. Figure 4 distinguishes these decisions along the path to a training batch.

4.2.1 Restart-state selection

The start distribution combines opening diversity with targeted exploration. Half of games begin after a uniformly selected zero to eight random legal plies. These prefixes diversify the opening cheaply; they are reconstructed in the recorded history but not searched or trained directly. Drawing zero plies also keeps some games at the ordinary initial position.

The other half begin from archived self-play states, falling back to random openings when a worker’s restart archive is empty. The archive favors positions with a meaningful un-

resolved alternative: they are not near the end of the source game or already decided, and search found a small set of plausible moves. A restarted game chooses an untried branch, reconstructs the prefix, then explores the alternative the source game did not play. Appendix D gives the eligibility thresholds.

The archive gives 30% probability to uniform selection. Otherwise it favors positions where search substantially corrected the network’s value estimate, while softening the priority so one extreme position cannot dominate. Age and capacity bounds remove old states, and exhausted positions leave the archive. Archives are local to workers.

Starting from archived states follows the search-control idea studied by Trudeau and Bowling [5]. Here, observed search disagreement and untried branches guide the archive. Playing those branches creates new trajectories; the replay sampler below instead chooses which existing positions recur during training.

4.2.2 Replay admission

Replay admits searched positions, including a final search used to evaluate a capped game’s endpoint. Random opening moves and reconstructed restart prefixes have no search target and are excluded. Admission does not depend on ply: searched endgame positions remain eligible throughout the game. Sparse policy storage retains at most 60 actions per position and records the discarded visit mass.

4.2.3 Replay capacity and reuse

A finite replay window trades diversity against target age. Small windows rapidly replace early policies but discard infrequent openings and endgames; larger windows preserve that coverage at the cost of retaining older targets. The retained schedule expands the active window as self-play supplies new positions.

The physical memory map is preallocated once, while its logical capacity grows from 600,000 to 20 million rows. The early window stays small when little data exists; later growth preserves more openings, endgames, and policy history. In

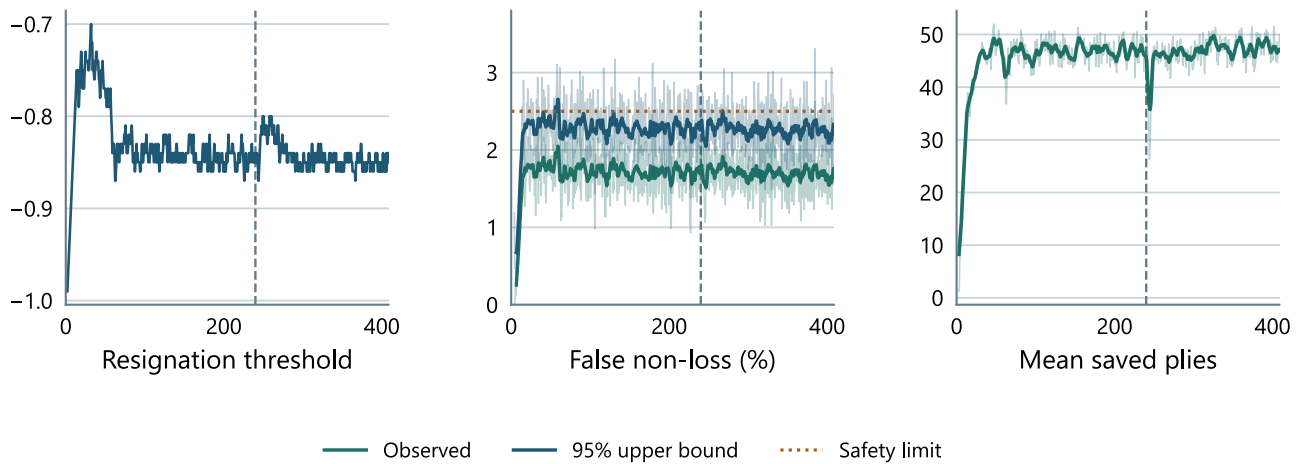


Figure 6: Resignation threshold, observed false-resignation rate and its one-sided 95% upper bound, and estimated saved plies during final training. The error panel shows periods when the resignation gate is enabled; the dotted line marks the 2.5% calibration target, including observed excursions. Saved plies are estimated from continuation games. Faint traces show raw values, with eleven-quantum averages overlaid. All horizontal axes show completed optimizer steps in thousands; the dashed vertical line marks the small-to-medium model transition.

an earlier campaign, playing strength continued to improve after fixed-dataset policy accuracy largely saturated, reinforcing the value of fresh and varied positions. Appendix D gives the full capacity schedule. Figure 5 shows replay occupancy and sampled-position age during final training.

Replay reuse introduces a second tradeoff. The configured ratio is the number of optimizer presentations funded by each newly admitted row. In the retained setting, four presentations are credited per row; a 500-step quantum at a global batch of 2,048 therefore requires 256,000 newly appended rows. Only positions that have actually reached the replay store count towards this allowance.

Higher reuse funds more updates from each game; lower reuse gives each update fresher positions if the actors can supply them. Short controls at ratios 4, 6.25, and 8 showed similar strength despite different update rates. The selected ratio of 4 favors freshness. Appendix C gives the duration and scope of these short controls.

4.2.4 Policy-surprise sampling

Search disagreement suggests an opportunity to learn beyond the network’s initial move preference. Within the live window, the sampler therefore favors rows with high *policy surprise*: divergence between search’s visit distribution and the network prior. Seventy percent of draws follow this signal, capped at 2.0 so extreme rows cannot dominate; the remaining 30% are uniform to preserve coverage. A row can recur across optimizer steps but is drawn only once within a global batch. This is related to prioritized replay [4], though it uses a different signal.

The priority deliberately changes which positions dominate training. It is not corrected back to uniform sampling. Because surprise can also reflect search noise or old targets, the uniform component preserves broader coverage.

Priority acts through sampling frequency, not through per-example loss weights: every sampled row has weight 1.0. An earlier replay design instead weighted merged duplicate positions by their multiplicity.

4.2.5 Resignation calibration

Resignation can save a large amount of search, but a false resignation turns a drawable or winning position into an incorrect terminal label. A fixed value threshold was therefore replaced by continuing measurement. Twenty percent of games are designated at creation as continuation games and can never resign. They reveal the outcomes of hypothetical triggers for thresholds from -0.99 through -0.70. A trigger requires both the root value and the best visited child’s backed-up value to cross the threshold; evidence from capped games is excluded because their natural outcome remains unknown.

Calibration estimates the fraction of hypothetical resignations whose continued games end in a draw or win. A threshold requires at least 100 observations and a one-sided 95% upper confidence bound no greater than 2.5%. It can become more conservative immediately but relaxes by at most 0.01 per publication. Recalibration tracks changes in the model’s value predictions. Figure 6 reports the threshold, rolling error estimates, and saved plies.

4.2.6 Outcome targets at the ply cap

A ply cap creates a harder target problem because there is no observed result at all. The project compared material heuristics, raw network value, values from adjacent searches, and a fresh search at the cut position. Playing games beyond the normal cap supplied later outcomes for evaluation. Brier score measures squared probability error, while cross-entropy penalizes assigning low probability to the eventual outcome; lower is better for both. At the earlier measured checkpoint, the cut-position search achieved Brier score 0.444 and cross-entropy 0.756, compared with 0.491 and 0.851 for material divided by 39. At the later checkpoint the corresponding scores were 0.193 and 0.374 versus 0.388 and 0.707. Search-root sign accuracy exceeded 98% in both cohorts; calibration, not merely sign, distinguished the targets. The retained worker therefore performs one full search at the actual cut position and uses its root value as the bootstrap. The root value becomes a soft win/draw/loss target, then materialization ap-

plies the configured per-ply blur. Appendix D gives the conversion.

4.2.7 Discounting and intermediate value targets

Discounting was introduced to favour faster conversion when many games approached the 250-ply cap. Search multiplies backed-up values by 0.99 per ply, reducing the value of a more distant win relative to an otherwise equivalent nearer win. Training applies a separate factor of 0.998 per remaining ply by blending the outcome’s WDL target towards uniform. This attenuates the expected value of distant outcomes and also reduces the certainty assigned to early positions. With signed values, discounting likewise makes a delayed loss less negative.

The two discounts act at different points in the learning loop, but share the intended incentive to complete winning games sooner. Their independent benefit was not established. Restoring searched endgame positions to replay, rather than discounting, was the identified repair for the conversion failure discussed in Section 6.1. Whether either discount improves the retained recipe remains open.

A separate scheduled blend incorporates up to 10% of the stored search-root value into the training target, combining the completed game’s outcome with search’s assessment of the current position.

4.2.8 Auxiliary trajectory targets

The completed trajectory also provides two auxiliary tasks: predict the next searched policy and the remaining game length. These encourage the shared representation to capture how play develops, not just the current move. Label availability depends on the recorded trajectory. A next-policy label exists only if there is a following searched observation, and a capped game cannot reveal its true remaining length. Missing labels are excluded from the corresponding loss.

Broader auxiliary bundles were set aside while diagnosing several simultaneous training problems, not because a controlled ablation found them harmful. Future-derived labels must come from complete trajectories and be masked whenever the future is unknown.

4.2.9 Reanalysis and publication cadence

Reanalysis refreshes stored policy targets by searching their positions with the latest network. It competes for compute with fresh self-play, which supplies new positions as well as outcomes. A bounded synchronous implementation was integrated into an earlier replay design but was not retained when that design was replaced. Its learning return relative to fresh self-play was not measured.

Publishing the latest model more often can also make targets fresher, because self-play then uses more recent predictions. Doing so every 100 optimizer steps spent too much time saving, exporting, checking, and activating models. The retained 500-step interval reduces that overhead while still refreshing the actors regularly.

4.2.10 Actor-trainer overlap

The actors and trainer share the GPUs. Pausing all actors gives the trainer more compute, but no new games arrive;

keeping every actor running slows training. The retained compromise keeps half the actors active during each 500-step training block. Models are still published at coordinated boundaries rather than changing underneath an ongoing search. Chapter 5 compares how much search and training this overlap supports.

4.2.11 The retained data strategy

The retained strategy combines broad opening coverage with targeted restarts, an expanding replay window, and policy-surprise sampling. Fresh-game supply limits reuse, while calibrated resignation and searched cutoff values reduce the cost of completing trajectories. These mechanisms control the distribution, age, and reliability of the training data; their combined effect, rather than replay volume alone, determines its value to the learner.

4.3 Choosing what the network predicts

Network architecture constrains both prediction quality and the volume of search affordable during training. We compared policy representations, convolutional and attention trunks, global context, and value heads, then examined model growth and quantization. The policy head was a major source of parameter cost in small networks; Figure 7 summarizes the three representations evaluated.

4.3.1 Three policy representations

All three policy families were implemented against the chess move interface, whose selected encoding contains 1,880 canonical actions. Controlled fitting experiments compare held-out policy cross-entropy in nats.

The **dense head** flattened a small spatial projection into 1,880 logits. Dense heads trained successful models, but on a small trunk their projection could dominate the parameter count. Variants changed projection width, spatial reduction, and final-map rank. A rank-96 variant reduced head size from roughly 484,000 to 207,000 parameters.

The **move-plane head** represented sliding directions, knight moves, and promotions as spatial move types. It could preserve board structure, but its experiments did not show an advantage over the dense control. In a short supervised comparison, the repaired plane head reached 2.1828 held-out policy cross-entropy after about 2,500 steps versus 2.0824 for the dense control, while its curve was still improving faster. An online plane-head trial was retired after slower learning and weaker play.

The two implementations differed: one gathered 1,880 canonical logits from 76 planes (56 sliding directions, eight knight moves, and 12 promotions); the other exposed all 4,864 plane-square cells and normalized only legal moves. Both express the same basic idea: predict move types at board locations rather than score a flat action list.

The retained **from-to head** uses query-key dot products over the 64 trunk squares, with a fixed gather into the canonical action space and separate promotion offsets. This retains spatial structure without a large dense projection. On the controlled convolutional trunk, the head used 51,072 parameters rather than 483,680 for the dense alternative. Section 3.5 and Appendix D specify its representation and action mapping.

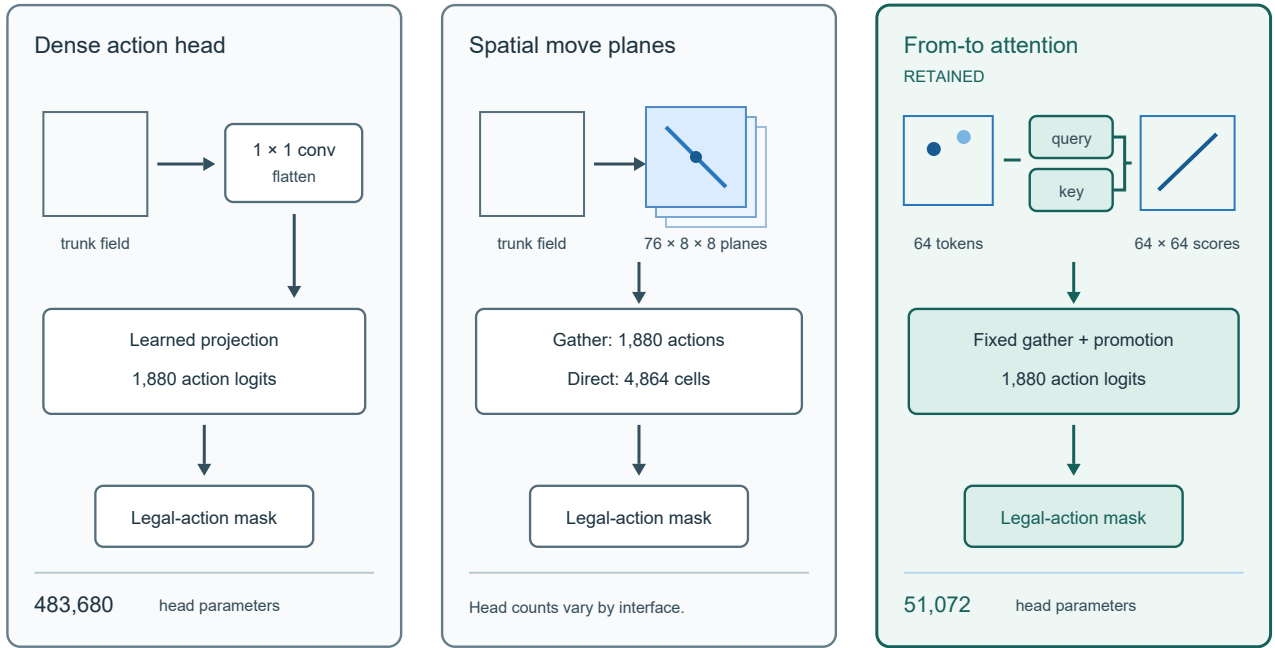


Figure 7: Dense heads project to a move list, move-plane heads predict spatial move types, and the retained from-to head scores square pairs before gathering canonical actions and masking illegal moves.

Holding that trunk fixed, the from-to head improved the held-out policy gap by 0.0298 nats (paired 95% interval 0.0285–0.0311). Spending the saved parameters on a wider trunk added only 0.0018 nats in the three measured cells; the missing fourth cell prevents fully separating the two effects. Forward throughput fell by approximately 1.9% at batch 512 and 9% at batch 64. The from-to head therefore improved policy fit and reduced parameter count, with a serving penalty that was smaller at the large batches used for self-play.

4.3.2 Board input and rule state

The selected model receives 52 side-to-move-canonical board planes: pieces, castling rights, en passant, checks, repetition, the eight most recent moves, material counts, and the fifty-move counter. Rule-sensitive planes keep positions with different legal or draw states distinguishable. [Appendix D](#), Table D1 lists all 52 planes and their encodings. File reflection is the only augmentation; it also mirrors action targets and exchanges kingside and queenside castling planes. Some earlier component comparisons used a 29-plane input, so their absolute scores are not input-matched to the final model.

4.3.3 Convolution, attention, and global context

The trunk comparison evaluated whether attention improved board-wide feature learning enough to justify its serving cost. Both designs shared their trunk across policy, value, and auxiliary heads. Because dense primary and auxiliary policy heads could dominate a small model’s parameters, head choice had to be controlled alongside the trunk.

The attention alternative treated the 64 squares as tokens, with learned row and column embeddings, pre-normalized self-attention, and GELU feed-forward blocks. No-bias, relative-offset, and input-dependent Smolgen-style attention biases were implemented. These biases respectively add no

positional preference, depend on the offset between squares, or depend on the board itself. The preserved comparison covers the first and third.

With bootstrap policy shape, head, runtime, and precision controlled, the convolutional trunk beat bare attention by 0.0060 nats on held-out teacher data. Smolgen gave the best attention cell, but changing the attention model’s dense head to from-to improved the held-out gap by a much larger 0.1573 nats. Attention also consumed more memory and served more slowly at the relevant batches. The tested convolutional design was the better choice for this workload; the result does not rule out attention in other chess systems.

Local convolution receives global context every second residual block: board-wide means and maxima from one quarter of the channels are projected back as biases on local features. Squeeze-excitation was another implemented context mechanism, using board-wide information to rescale channels. Global pooling appeared to learn faster, with no clear final-strength difference; this remains a qualitative observation because the comparison results are unavailable. The pooled features give local convolutions access to the whole board, following the KataGo precedent [2]. Those context operations remain in floating point when the main convolutions are quantized.

4.3.4 Value and training-only heads

The outcome head predicts win, draw, and loss rather than a single scalar. Search converts this distribution to an expected value when necessary, while training and diagnostics retain the draw probability. A compact two-channel spatial reduction and 48-unit hidden layer was retained. A matched 32-channel probe added 97,020 parameters and reduced measured training throughput by 1.31%, while improving total loss by only 0.00309 in one short seed and slightly worsening WDL loss. This justified keeping the smaller head. The

earlier scalar-to-WDL transition has no preserved isolated strength comparison.

Training also predicts the next move’s searched policy and normalized remaining game length, with loss weights 0.15 and 0.1. The next-policy target uses the following state’s own side-to-move action space, legality mask, and symmetry transformation. Both heads train the shared representation without making inference more expensive, because deployment removes them. Other auxiliaries were set aside while debugging potential interference, not because they were shown harmful. The retained pair has not received a long matched self-play ablation.

4.3.5 Bootstrap and optimization controls

Initialization shapes the first search priors and, through them, the first self-play targets. In an initial architecture comparison, the attention policy put only about 0.11 probability mass on its top three moves while the convolutional control was effectively one-hot. Neither extreme represented chess knowledge, but each induced different data. The retained bootstrap uses architecture-appropriate initialization, deterministic construction, a small final policy projection, and calibration on 516 encoded positions toward a common policy concentration. Those positions set numerical scale without supplying supervised chess targets.

The retained optimizer is Nesterov SGD. Frozen-replay tests helped choose warm-up and learning-rate settings before committing to self-play runs. AdamW had also trained successful models; the choice of SGD does not mean AdamW failed. Gradient clipping limits unusually large updates. The final run completed 408,500 optimizer steps, with the learning rate, losses, and gradient norms shown in Appendix A.

4.3.6 Quantization as an architectural constraint

INT8 deployment required changes to the residual architecture as well as quantization-aware training (QAT). The experiments compared prediction fidelity and compiled throughput: an architecture that tolerates rounding but introduces expensive precision conversions may still be unsuitable for self-play.

Post-training quantization of the ordinary residual tower accelerated inference but severely distorted predictions. Activation ranges grew from about 0.78 near the input to roughly 40–43 late in the network; depending on calibration, full-trunk INT8 preserved only 13.3–25.9% policy top-one agreement. Weight-only quantization was faithful but slower than TensorRT FP16. Quantization therefore became a network-design problem rather than a final export switch.

A scaled pre-activation block bounded residual activation ranges, but its normalization, clipping, scaling, and repeated precision conversions fragmented the compiled TensorRT graph. It expanded an 83-layer FP16 graph to 330 layers and an INT8 graph to 470 layers, with many reformats, yet still failed fidelity. The retained scaled post-activation block instead keeps the conventional sequence of convolution, normalization, capped activation, convolution, normalization, scaled residual addition, and capped activation. Activations are capped at six, and residual branches are scaled by the inverse square root of depth. The scale can be folded into the second convolution at export, which preserves more efficient compiler tactics.

Batch-normalization folding changed the quantization behaviour as well as the compiled graph. The final run keeps the trainable model unfused and performs folding and recalibration on a deployment copy. Separate fitting experiments also evaluated continued training in the folded topology.

The scaled post-activation block learned normally under quantization-aware training. After continuation in the folded deployment topology, a production-sized smoke test reached roughly 135,000 INT8 positions per second, compared with 60,000 for TorchScript BF16 and 99,000 for TensorRT FP16. These are model-core rates, not end-to-end self-play rates. Folding only after training damaged agreement, and global context and the heads remain outside the INT8 trunk. The result is evidence for an architecture compatible with efficient INT8 inference, rather than a playing-strength advantage over the ordinary floating-point residual block.

4.3.7 Progressive model sizing

Following KataGo [2], progressive sizing uses a small network for early self-play while a larger candidate trains on the same replay before promotion. The throughput advantage is most useful while the smaller network has enough capacity to absorb the available experience. The small-to-medium transition worked repeatedly in this project.

Candidate start follows a stage-specific searched-Elo plateau; promotion instead requires two passing paired matches against the active model. Section 6.3 explains why matches replaced training loss as the promotion criterion; Appendix D states the retained thresholds.

An independently initialized larger candidate needed substantial catch-up. Function-preserving growth instead initializes the larger model to compute the same predictions as the medium model. It avoids relearning that function, but might also bias optimization toward the smaller model’s existing representation. The limited grown-model continuation reached parity without a clear strength gain. The reported model therefore remains medium-sized; choosing between catch-up and growth needs a matched-compute comparison.

4.3.8 Distillation and compact models

Distillation evaluated the tradeoff between reduced inference cost and approximation of a stronger teacher. We tested two sources of supervision: the larger teacher’s direct predictions and the targets already collected in replay. In the first, students learned the teacher’s legal-move probabilities and WDL predictions from games played without search. Increasing this dataset from one million to six million positions mattered more than a small capacity sweep. The strongest 1.33-million-parameter student trailed its teacher by 176.1 Elo at 25 searches each and by 38.4 Elo when it received the measured shallow equal-compute allowance of 58 searches. At 250 searches each the gap widened to 257.6 Elo. Deeper search amplified the better prior rather than washing out approximation error. Fixed-ply sampling gave the dataset only one side-to-move parity, and the teacher had a known conversion weakness, so these absolute gaps do not transfer to the final model.

Replay-target compression instead trained on sparse MCTS visits, outcomes, root values, and replay metadata from a frozen ten-million-row window. The selected 474,069-

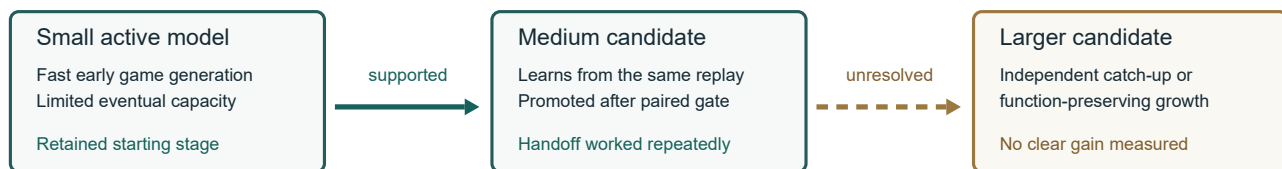
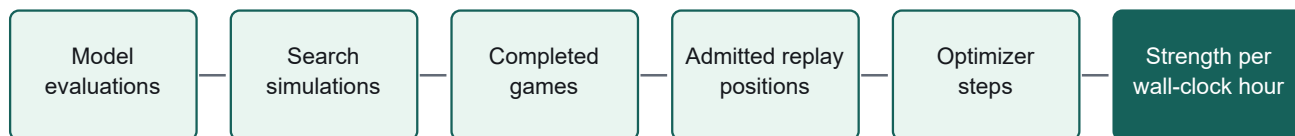


Figure 8: A small model reduces early self-play cost, and a medium candidate trains on the same replay before paired-match promotion. The larger candidate may avoid catch-up with function-preserving growth, but the limited continuation did not demonstrate a strength gain; the reported checkpoint remains medium-sized.



A local speedup matters only when it reaches the rightmost outcome.

Figure 9: Search speed passes through game completion, replay admission, and optimizer work before it can affect playing strength. Each boundary has its own throughput measure.

parameter student was 13.20 times smaller than its teacher. It trailed by 291.3 Elo at 64 searches each and by 166.2 Elo when its measured saturated serving advantage allowed 186 searches against 64. It reached statistical parity only at an equal-multiply-accumulate allowance of 850 searches, which counted neural arithmetic but omitted tree work and inference overhead. The smaller model’s theoretical compute advantage was therefore much larger than the extra search it could actually perform in the same time.

A final compression study trained a 470,295-parameter student on a separate frozen 20-million-row replay snapshot for 110,000 optimizer steps, roughly 23 epochs. It reached approximately 2,700 benchmark Elo at 10,000 searches and 2,870 at 100,000 searches against the one opponent tested there. Chapter 8 presents these final student results alongside the full model.

The resulting compact models reduced inference cost, but their realized search advantage was substantially smaller than their parameter or arithmetic ratios. The teacher’s strength advantage also increased with search depth in the measured comparisons. Distillation was separate from the primary self-play training run.

4.3.9 Decision

The retained network combines the rule-complete 52-plane input, a shared convolutional trunk, periodic global context, the from-to policy head, a compact WDL head, and training-only next-policy and remaining-length objectives. Scaled post-activation blocks make the trunk compatible with quantization-aware deployment. Progressive sizing successfully exploited a small model before handing off to the medium model, while the value of the larger stage remains unresolved. Component tests support parts of this design, while final playing strength belongs to the assembled system.

5 From inference speed to learning speed

Under a fixed compute budget, inference throughput matters through the training data it makes affordable. Search, game

completion, replay delivery, and optimization jointly determine that supply (Figure 9). This chapter examines the bottlenecks at those stages and the scheduling required to share GPUs between actors and the learner. Relative gains are reported within each controlled benchmark; [Appendix C](#) collects the absolute rates and settings.

5.1 Native ownership of the search loop

Per-position Python dispatch made board updates, legal-move generation, encoding, and inference submission a host-side bottleneck. The production loop therefore keeps game rules, search trees, selection and backup, and asynchronous inference requests in C++. Python coordinates configuration, replay, training, publication, and evaluation outside that hot path.

Each actor interleaves 512 games through a preallocated inference pipeline and retains search subtrees across moves. The retained topology uses four actors per GPU, one inference worker per actor, batches of up to 320 positions, and two outstanding batches. This provides independent work to overlap tree traversal with neural evaluation without introducing a Python boundary at each leaf.

5.2 Batching and host-side submission

Batching amortizes GPU launch costs, but sustained utilization also requires timely host-side preparation. Preallocated staging buffers, asynchronous copies, and completion events overlap encoding and submission with GPU execution. CUDA graph replay reduces the remaining launch overhead.

In a controlled 32-process self-play workload, the optimized path increased search throughput by 20.5% while reducing aggregate actor CPU consumption from 52.6 to 19.8 cores. Average inference batch size rose from 141 to 222 and the number of model calls fell by 24%. On the same node restricted to 24 CPU cores, throughput more than doubled because reducing submission overhead allowed the previously starved GPUs to remain busy. The larger gain under the CPU quota identifies host submission as a substantial constraint on GPU utilization.

Full batches were necessary but insufficient for saturation. Reducing TensorRT actors from four to two per GPU lowered search throughput by 39% despite full individual batches, because fewer actors reduced overlap between CPU preparation and GPU inference. Additional inference threads instead duplicated CUDA contexts and fragmented batches. The measurements supported process-level concurrency with one inference thread per actor.

Per-tree parallelism can supplement batching when too few independent games remain active. Unlike inter-game concurrency, it changes leaf selection through virtual reservations and can reduce search quality. Its strength-throughput trade-off is examined in Chapter 4.

5.3 Inference runtimes and precision

With host submission sustained, runtime and precision determine the cost of evaluating each batch. A TensorRT FP16 engine delivered 1.86x the inference throughput of a TorchScript BF16 control in a matched benchmark. Quantization-aware INT8 added a further 1.31x over TensorRT FP16 on the tested quantization-oriented network. Production-topology tests found INT8 gains of 14.4% for the smaller network and 39.1% for the medium network.

As Section 4.3 explains, the INT8 speedup required training the network to tolerate quantization. Most trunk convolutions run in INT8, while the start block, heads, and linear layers remain at higher precision. Export records the quantization and dequantization operations explicitly in ONNX so TensorRT can compile the intended arithmetic. Rather than rebuild the complete engine after every update, publication refits a prepared template with the new weights [8]. Chapter 6 examines a failure in this step that made output comparisons essential.

The alternative `torch.compile` path accelerated eager batch-64 inference by roughly 27–33%, but fused TorchScript remained faster. In the tested eight-GPU training workload, compilation reduced throughput by about 18% relative to eager execution, while `bfloat16` autocast improved it by 9.2%. Compilation was not retained for production inference or training on this workload.

Similar parameter counts did not imply similar inference cost. Width and depth changed kernel efficiency, TensorRT tactics, and memory behavior discontinuously. Channels-last layout and cuDNN autotuning helped relevant CNN shapes, but no analytic parameter-count rule predicted the fastest network. Progressive model sizes were therefore benchmarked at their actual serving batch and precision rather than selected from FLOPs alone. The width and depth sweeps in [Appendix C](#), Tables C1 and C2, quantify this mismatch: narrower networks were not consistently faster, and changing batch size could reverse the ranking of deep-narrow and shallow-wide designs.

5.4 Replay materialization and training supply

Replay delivery must sustain both trajectory ingestion and training-batch retrieval. Parallel materializers convert completed trajectories into a circular memory-mapped store, avoiding repeated trajectory decoding in the training path. Direct column access and pinned-memory prefetching reduce batch preparation and transfer costs.

Table 3: Actor overlap: optimizer throughput and concurrent search

Actors	Train (k/s)	Search (k/s)	Cycle (s)
8	21.5	506	117
16	17.1	606	113
32	9.21	742	111

A loader benchmark on 2.5 million rows became 8.26x faster after compacting 5,000 small producer shards into 25 containers, putting delivery capacity 44% above the measured trainer demand. In a live interval, materialization could append positions more than six times as fast as self-play supplied them. The replay pipeline therefore had enough headroom to keep up with game production.

The configured reuse ratio couples optimizer progress to newly admitted positions, so loader capacity beyond trainer demand does not by itself increase training volume. Persistent distributed trainer processes avoid startup and model construction costs between blocks. The retained global batch of 2,048 uses `bfloat16` autocast across eight GPUs. Larger batches improved hardware throughput in the benchmark but also changed the number of optimizer updates per training position, so batch size was selected as part of the learning recipe rather than for throughput alone.

5.5 Overlapping self-play and training

Self-play and training compete for GPU capacity but have different resource profiles. Search includes CPU traversal and transfer intervals that allow useful overlap with optimizer work. Scheduling must therefore balance the slower training block against the reduction in subsequent waiting for new games.

The overlap sweep compared keeping 8, 16, or all 32 actors active during training, measuring both training time and the remaining wait for self-play. Moving from half to all actors active nearly doubled the trainer’s work time for only a small reduction in the complete cycle. The retained half-active policy balances ongoing game production against optimizer throughput; Table 3 gives both sides of this tradeoff. Complete-cycle time includes training and the remaining wait for self-play data; with all actors paused, training alone reached 25.3 thousand samples/s.

The useful overlap fraction depends on search cost. As visit budgets and model size rise, self-play becomes more expensive relative to training; actor settings measured at the beginning cannot simply be extrapolated to later stages.

5.6 Throughput allocation within the learning loop

The resulting system allocates throughput across data generation and optimization rather than maximizing either in isolation. Native search and batched inference increase game supply, replay materialization prevents delivery from limiting training, and actor overlap replenishes replay during optimizer updates.

Additional search capacity can support more games at a fixed budget or deeper searches per position. Those uses alter data diversity and target quality differently, while replay reuse controls their rate of consumption. The systems improvements therefore expand the feasible training regime; the recipe determines how that capacity is spent. Chapter 7 evalu-

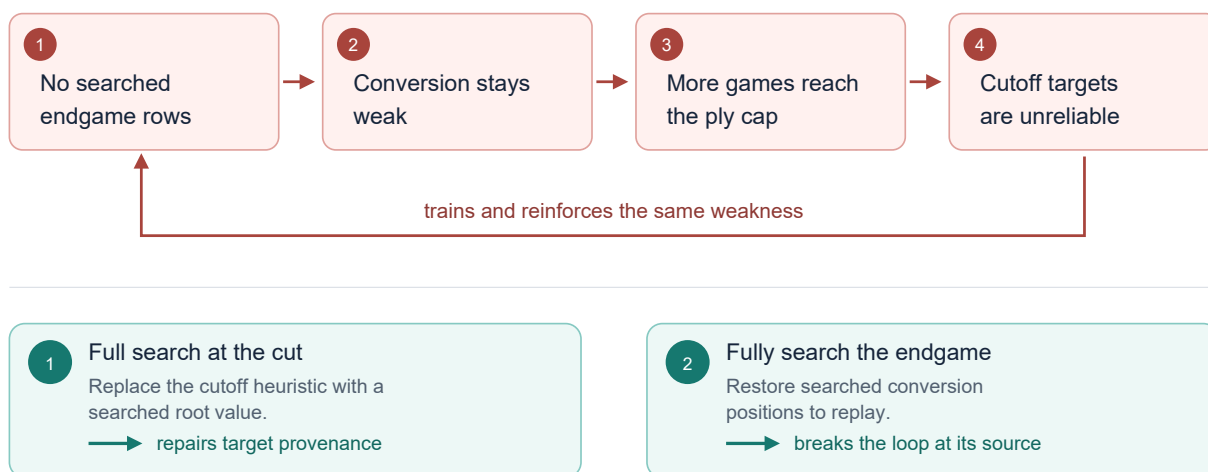


Figure 10: A searched cutoff value improves labels for unfinished games; restoring fully searched endgames also returns the examples needed to learn conversion.

ates the resulting progress in playing strength over wall-clock time.

6 Three failures that changed the method

Three failures exposed dependencies that local optimization metrics did not capture: endgame sampling altered the reliability of value targets, TensorRT refitting changed the deployed policy, and loss-based promotion confused replay fit with playing strength. Their causes and corrections illustrate why data generation, deployment, and model selection must be evaluated as parts of the learning process.

6.1 Late-game target poisoning

To prevent noisy positions from exceptionally long endings from dominating replay, an earlier self-play design excluded the late-game tail from primary training targets and reduced its search budget. This saved compute on positions that would otherwise be discarded, but systematically removed examples of endgame conversion. With little training on those positions and only shallow search during play, the model frequently failed to convert advantages before the ply cap. A heuristic cutoff estimate then supplied the outcome target for the preceding trajectory.

The interaction was self-reinforcing: inadequate conversion produced more capped games, whose unreliable targets further degraded the policy and value estimates needed to finish them. During the most affected interval, approximately one ply in seven was ineligible for training, 27–32% of games reached the cap, and 36–38% of admitted rows inherited a cutoff target. Consequently, an error introduced at the end of a game affected training throughout its recorded trajectory. Replay turnover removed the original rows but did not necessarily remove their influence on the model or the self-play distribution.

The correction addressed target estimation and training coverage separately. Replacing the material heuristic with the root value of a full search at the cutoff reduced Brier error from 0.491 to 0.444 and cross-entropy from 0.851 to 0.756 on 2,282 early-cut positions in a controlled continuation study. A later set of 1,144 positions showed the same ordering. Re-

moving the forced cheap-search tail then restored searched endgame positions to replay, allowing the network to learn conversion rather than merely receive a better estimate when conversion failed.

Figure 10 distinguishes these two interventions. Improving the cutoff target reduced label error; restoring endgame coverage corrected the sampling policy that sustained the failure.

6.2 Prediction drift under TensorRT refitting

The deployment pipeline refits a compiled TensorRT template after each training block to avoid rebuilding the engine. In a failing template, equal quantization-scale constants allowed optimizations that became invalid when subsequent training produced unequal scales. TensorRT accepted all replacement weights and reported a successful refit, yet the resulting engine no longer reproduced the source network’s predictions.

On 516 real positions, the faulty engine’s legal-policy KL divergence from the source network was approximately 1.05, and repeated refits with identical inputs changed individual logits by 10–15. Lowering the optimization level or making the template’s scale constants distinct before compilation reduced KL divergence to approximately 0.0012 and restored deterministic refitting. The defect required the combination of equal source scales, optimization level four or higher, and a subsequent refit with unequal scales.

Template construction now separates equal quantization scales before optimization and uses a lower default optimization level. Deployment validation compares top-one legal-move agreement, policy KL, and WDL error against the source network. These checks test the predictions consumed by search, independently of whether the compiler reports a successful build or refit.

6.3 Promotion from incomparable training losses

The progressive controller initially used smoothed training loss as a low-cost proxy for promotion readiness, avoiding additional searched matches. Extra catch-up training, however, gave the larger candidate more optimizer quanta on the same replay distribution. Its lower loss therefore reflected unequal training exposure as well as model quality. The gate pro-

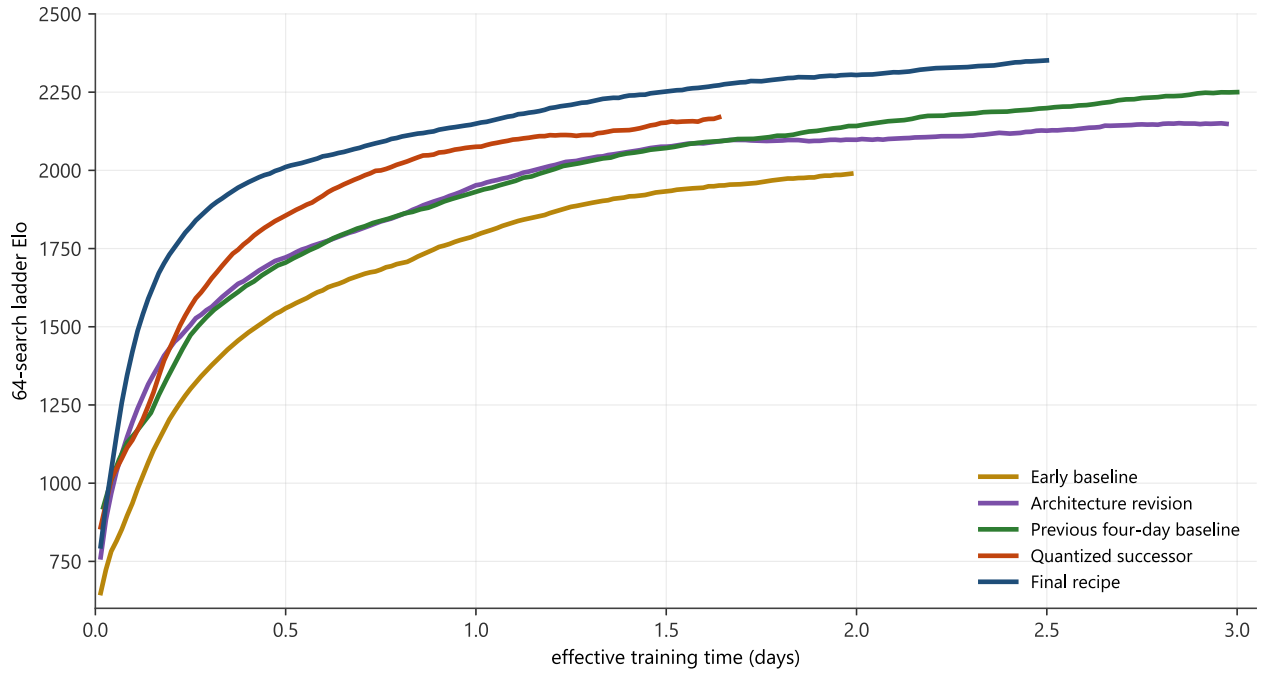


Figure 11: Smoothed 64-search training ladders across five chess campaigns. The final curve ends at 2.5 days and the preceding baseline at 3 days.

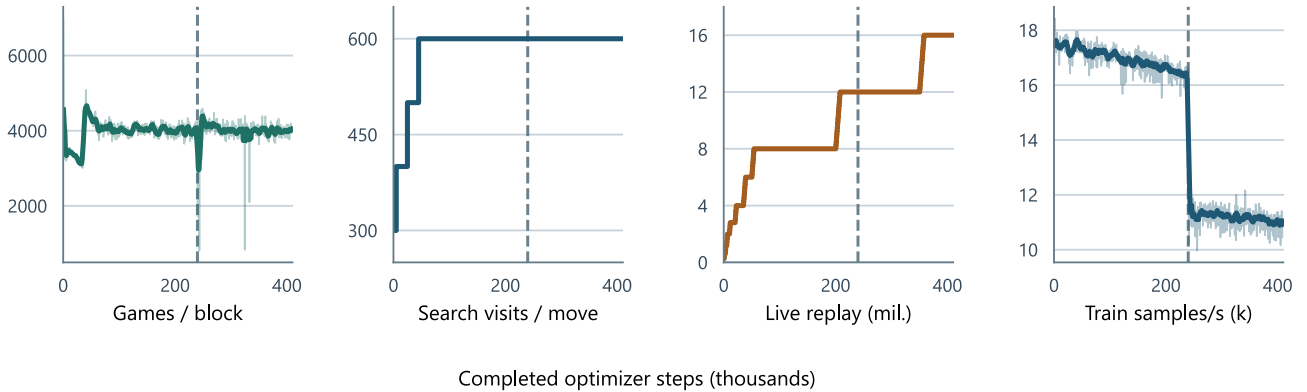


Figure 12: Training volume and throughput on a common optimizer-step axis. Panels show completed games per training block, search visits per move, live replay occupancy, and trainer throughput. Replay capacity increases in steps, while the small-to-medium transition coincides with lower trainer throughput.

moted a candidate approximately 270 Elo weaker than the active player, despite its deployment artifact passing inference-fidelity checks.

Promotion now requires a score of at least 0.48 in two consecutive paired matches against the active deployment artifact. A failing score resets the sequence; failed or cancelled matches do not count. This separates the optimization objective from the replacement decision: training loss guides candidate fitting, while direct play determines whether it can replace the active model. Appendix D specifies candidate timing and catch-up training.

7 Training progress under limited compute

The final training run produced a 6.32-million-parameter chess model in 2.5 days on one node with eight RTX 4070 SUPER GPUs and 80 logical CPUs. This chapter examines the progression in strength and the training volume made possible

by the integrated system. The complete recipe is specified in [Appendix D](#); `chess-final-config.yaml` [10] remains the entry point for reproduction.

7.1 Progress across training campaigns

Figure 11 compares the 64-search training ladders across five chess campaigns. The final recipe reached a higher plateau within a shorter training window than the preceding baseline. Its training-ladder rating rose from about 800 to 2,370, with a peak near 2,410.

The larger progression across the plotted campaigns reflects successive changes to architecture, data generation, and training efficiency.

7.2 Training volume and model transition

Table 4 summarizes the scale of the final run.

The search estimate uses a conservative 100 searched plies per game and roughly 600 simulations per move.

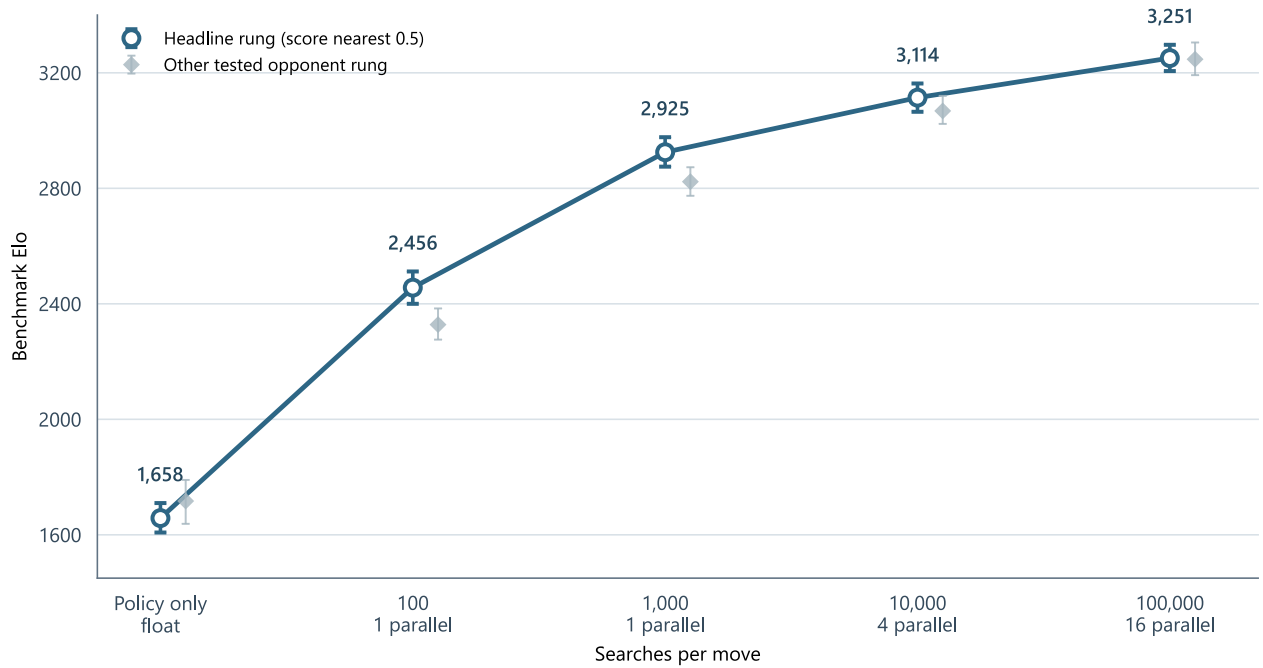


Figure 13: Final playing strength across search budgets. Connected points use the opponent rung with score nearest 50%; pale diamonds show the second opponent-based estimate and bars indicate 95% match-bootstrap intervals. The horizontal axis lists search budgets categorically.

Table 4: Final training run

Quantity	Final training
Duration	2.5 days
Model parameters	6.32 million
Completed games	3.25 million
Mean game length	~104 plies
Estimated search simulations	~195 billion
Materialized positions	209 million
Training presentations	837 million
Optimizer steps	409 thousand
Live replay at selection	16.0 million positions
Node rental cost	\$43.2

Figure 12 relates this volume to replay growth and the transition from the 12×128 to the 14×160 model. The smaller network supported faster early training; median trainer throughput fell from about 17.0 thousand to 11.2 thousand samples/s across the two stages. Search increased from 300 to 600 visits per move over the plotted run. The configured 800-visit stage lies beyond the selected checkpoint.

The expanding replay window retained a broader history of self-play while the learner received approximately four presentations per admitted position. Throughput, search budget, and replay growth therefore changed together as training progressed. Loss, learning-rate, and gradient diagnostics are provided in Appendix A.

The reported checkpoint was selected near the strongest region of the training ladder. A function-preserving 19×176 continuation recovered parity but did not establish a higher plateau.

8 Playing strength and model compression

The final model reached approximately **3,250 benchmark Elo at 100,000 searches per move**, corresponding to roughly five seconds of thinking time on an RTX 4070 SUPER. This chapter examines the strength gained from search and the performance retained by a much smaller distilled model. Table 1 contains the full paired-match results; Appendix B specifies the evaluation protocol.

8.1 Strength across search budgets

Search increased playing strength from approximately **1,660 Elo for policy-only play** to **3,250 Elo** at the deepest budget, a gain of about 1,590 points. Figure 13 shows continuing improvement with diminishing returns: the final tenfold increase in search added roughly 140 Elo, substantially less than the increases at shallow budgets.

The two opponent-based estimates at the deepest budget differ by only four Elo. At shallower budgets the estimates are farther apart, as shown by the paired points. Appendix B describes the calibration and rating calculation.

Deeper searches use additional leaf parallelism to improve GPU utilization. The evaluation curve uses one leaf at a time at 100 and 1,000 searches, four at 10,000, and sixteen at 100,000. Section 4.1 examines the strength–latency trade-off.

8.2 Distilling a smaller player

The distilled student has approximately 470 thousand parameters, **13.4 times fewer** than the teacher. It trained for 110,000 optimizer steps on a frozen 20-million-position replay snapshot. Increasing search from 10,000 to 100,000 raised its rating from approximately 2,700 to 2,870 Elo (Table 5).

Table 5: Distilled student strength across search budgets

Searches	Elo	95% interval
10,000	2,700	2,640–2,750
100,000	2,870	2,820–2,940

The small network retains substantial playing strength, but the gap to the teacher remains despite deeper search.

9 Limitations and open questions

The study demonstrates an integrated training recipe under a limited compute budget. Its principal limitations concern attribution of the gains, calibration of playing strength, and transfer to larger models or different workloads.

9.1 Attribution and scaling

Most component experiments used a single seed, a short continuation, or frozen replay. These controls supported design selection at affordable cost, but do not identify an independent final-strength contribution for each component. Policy-head and global-context comparisons also include qualitative observations rather than complete matched evaluations.

The attribution problem is strongest for coupled choices. Progressive sizing changes capacity and training history as well as self-play cost; replay growth and reuse alter both data exposure and demand for fresh games. Matched-compute experiments are needed to separate those effects.

9.2 Persistent plateaus and the remaining engine gap

The remaining gap to leading engines is estimated at roughly 400 Elo. Closing it requires more than improving the implementation of the existing recipe. Across the project’s training runs, playing strength repeatedly approached a plateau and did not escape it during extended training. These observations make additional hours under unchanged settings an unpromising route to a substantial gain.

Four changes are the most plausible next tests: substantially larger models to increase representational capacity; more search during self-play to improve targets; lower replay reuse to increase fresh-game exposure; and a larger replay window to retain more diverse experience. Each changes a different potential limit on learning, and each also changes compute demand or data age. They should be compared by strength gained at matched compute, not solely by training loss or update count.

9.3 Rating calibration and evaluation scope

The benchmark scale derives from a published calibration of fixed-node Stockfish 13 [9], rather than direct matches against unrestricted engines. Each final match contains 100 games, and the reported intervals capture match sampling uncertainty with the calibration anchors held fixed. Agreement between opponent rungs is close at the deepest budget and weaker at shallow budgets. A larger match set and broader opponent field would help distinguish sampling variation from calibration and matchup effects.

The search-budget curve combines deeper search with increased parallelism. It measures the deployed operating points, not the isolated effect of search depth. Backend differ-

ences likewise enter policy-only, searched, and student evaluations.

9.4 Workload dependence and development cost

The rejected optimizations remain plausible under other workloads. Diverse chess self-play produced too few exact graph or inference-cache hits to offset lookup and synchronization costs; repeated analysis may offer more reuse. In the GPU-bound training workload, actor-trainer overlap limited the cadence gain from adaptive stopping: reducing search work did not proportionally reduce the complete training cycle.

The \$43.20 rental cost covers the final training run, excluding development experiments and evaluation. It characterizes the cost of executing the recipe rather than discovering it. Reproduction also depends on hardware contention, drivers, and TensorRT versions; the configuration, published model, and evaluation protocol define the comparison, while exact weights and timing may vary across executions.

10 Conclusion

In 2.5 days on one eight-GPU node, an AlphaZero-style chess system trained from random initialization produced a 6.3-million-parameter model measuring **3,251 benchmark Elo at 100,000 searches per move** against a fixed-node Stockfish 13 ladder. Without search, the same model measured **1,658 benchmark Elo**.

The system combines a compact structured policy head and shared convolutional backbone with progressive small-to-medium sizing, targeted restart states, and prioritized replay. An INT8-capable architecture, native search, batched TensorRT inference, and distributed training supplied the required volume of self-play data and optimizer updates. The result demonstrates the strength achievable by this integrated recipe within the available compute budget.

The component studies also identify limits to local optimization. Better deep-policy fidelity from learned search allocation did not produce better online learning, and fewer simulations from adaptive stopping yielded little reduction in cycle time. Exact graph and neural-cache reuse did not offset their overhead. More seriously, removing searched endgames corrupted value targets, successful TensorRT refits altered predictions, and loss-based promotion selected a weaker player. These findings connect efficiency to the quality of the data and model actually consumed by self-play.

Further scaling remains open. The larger model recovered its parent’s strength without surpassing it during the available continuation, while distillation retained substantial strength in a much smaller network without matching the teacher. Repeated plateaus make longer training under unchanged settings an unlikely solution. Larger models, deeper self-play search, lower reuse, and broader replay are the next hypotheses to test against measured strength per unit of compute.

References

- [1] Silver, D. et al. (2017). *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. [arXiv:1712.01815](#).
- [2] Wu, D. J. (2020). *Accelerating Self-Play Learning in Go*. [arXiv:1902.10565](#).
- [3] Lan, L.-C. et al. (2021). *Learning to Stop: Dynamic Simulation*

Monte-Carlo Tree Search. [arXiv:2012.07910](#).

- [4] Schaul, T. et al. (2015). *Prioritized Experience Replay*. [arXiv:1511.05952](#).
- [5] Trudeau, A. and Bowling, M. (2023). *Targeted Search Control in AlphaZero for Effective Policy Improvement*. [arXiv:2302.12359](#).
- [6] Czech, J., Korus, P., and Kersting, K. (2020). *Monte-Carlo Graph Search for AlphaZero*. [arXiv:2012.11045](#).
- [7] KataGo project (release 1.17.1). [Self-play training methods](#).
- [8] NVIDIA. [TensorRT engine refitting](#).
- [9] Meloni, M. (2021). *Stockfish and Lc0, test at different number of nodes*. [Calibration benchmark](#).
- [10] Braun, B. [Advanced Techniques in Chess Engines: source code and release documentation](#).
- [11] Braun, B. [AlphaZero chess model artifacts and model card](#).
- [12] Braun, B. [Live chess demonstration](#).

A Training diagnostics

The diagnostic traces cover the training sequence that produced checkpoint 1026: 817 quanta of 500 optimizer steps at global batch 2,048, totaling 408,500 steps and 836,608,000 presentations. Checkpoint identifiers include additional experimental branches, whereas these plots exclude the reverted INT8 conversion, invalid capacity promotion, and subsequent larger-model continuation. Checkpoint 1026 was selected near the strongest 64-search ladder region and was the last checkpoint in that region with the complete model, optimizer, QAT, ONNX, and TensorRT artifacts retained.

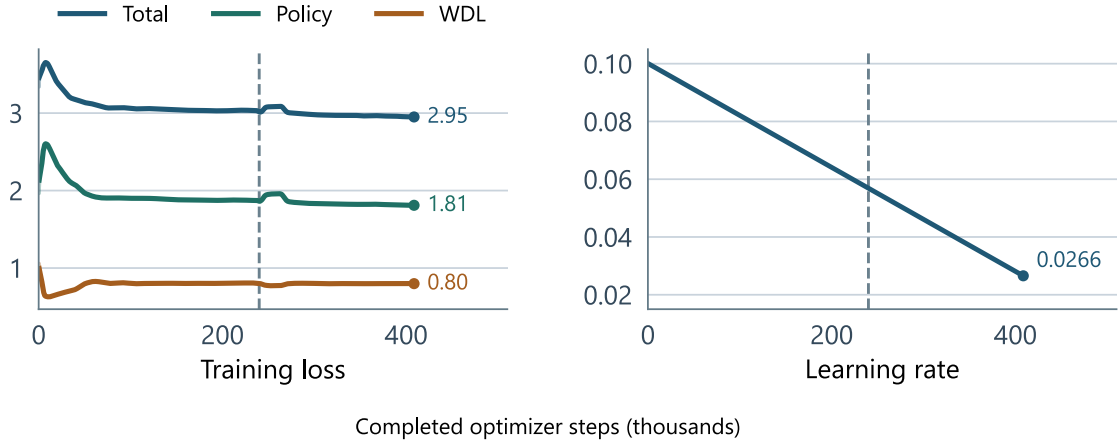


Figure A.1: Training losses and learning rate. Raw losses are overlaid with 11-quantum moving averages; the dashed line marks the small-to-medium transition at 240,000 optimizer steps. From 300,000 steps to selection, total loss decreases from approximately 2.98 to 2.95 while WDL loss remains near 0.80. The learning rate declines from approximately 0.10 to 0.0266. Labels identify the final values.

The exact counters underlying Chapter 7 are 3,249,647 completed games and approximately 209.15 million net materialized positions, with 16 million live rows at selection. Median trainer throughput was 16,977 samples/s over 480 small-model quanta and 11,194 over 337 medium-model quanta. Figure 12 in Chapter 7 shows these trajectories. Node rental was \$0.72/hour, totaling \$43.20 for the 2.5-day run.

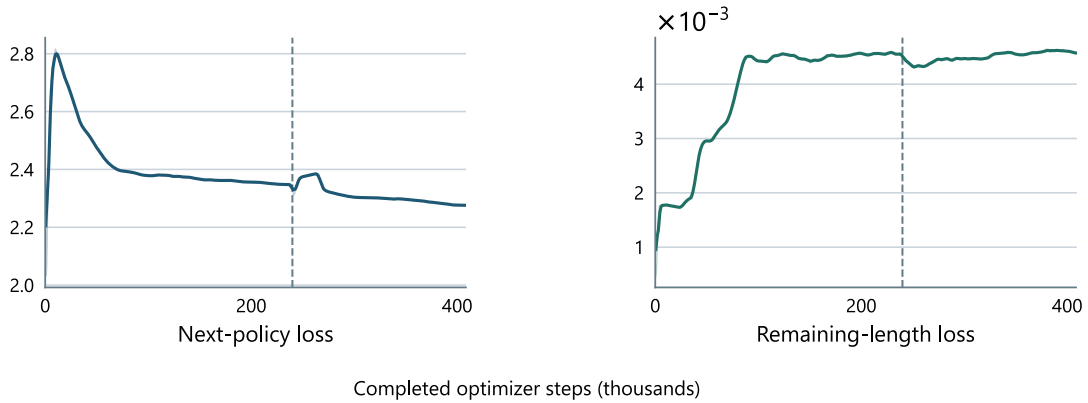


Figure A.2: Next-policy and remaining-game-length losses on separate scales. Faint traces show recorded values; solid curves show 11-quantum moving averages. The dashed line marks the small-to-medium transition, as in the remaining optimizer-step plots.

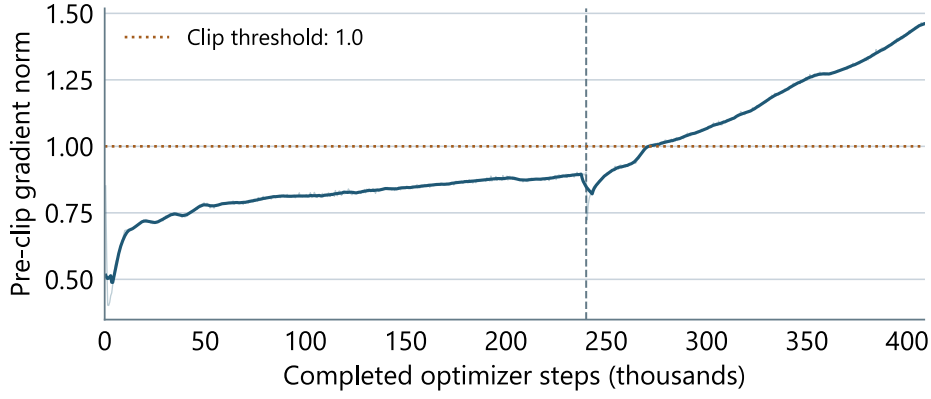


Figure A.3: Mean pre-clipping gradient norm in each 500-step training block. The dotted line is the configured norm limit of 1.0. The mean rises above that limit during medium-model training; the optimizer clips individual steps before applying them.

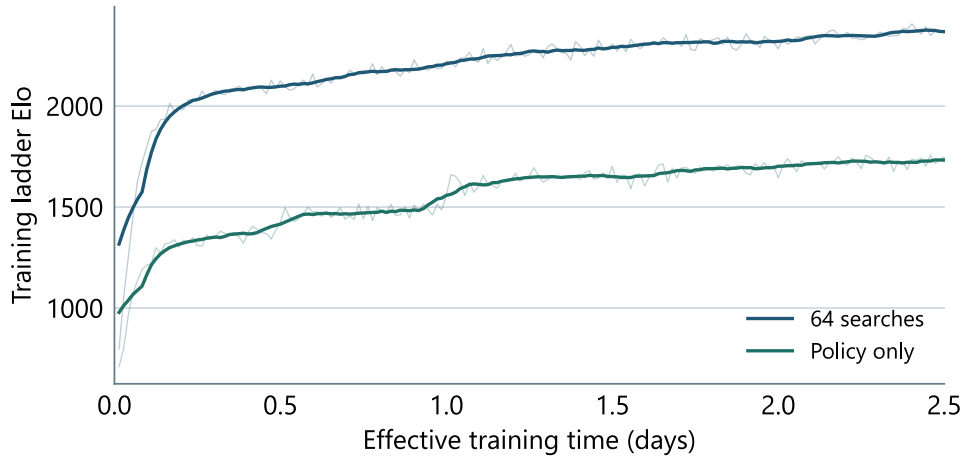


Figure A.4: Policy-only and 64-search training-ladder ratings over the final 2.5-day window. Faint traces show individual evaluations and solid curves average eleven evaluations. Both improve over training; Table 1 reports the separate final matches.

Replay occupancy, sampled-position age, and resignation calibration are shown with their corresponding methods in Section 4.2 (Figures 5 and 6).

B Evaluation detail

This appendix specifies the match protocol and rating calculation underlying the results in Chapters 2 and 8.

B.1 Fixed-node match protocol

Each of the ten teacher matches in Table 1 comprises 100 games from 50 openings played with colours reversed. Stockfish 13 used one thread and 1,024 MiB hash. Its fixed node limits were assigned the historical benchmark-Elo anchors of 1,700 at 1,000 nodes, 1,890 at 2,000, 2,220 at 5,000, 2,470 at 10,000, 2,700 at 20,000, 2,960 at 50,000, 3,100 at 100,000, and 3,230 at 200,000 [9]. These anchors were read from the published calibration curve. The reported rating, at each candidate budget uses the tested opponent against which the score is closest to 0.5. Searched games used the INT8 TensorRT artifact; policy-only games used the corresponding float TorchScript export.

For score s against an opponent anchor R , benchmark Elo is $R + 400 \log_{10}(s / (1 - s))$. Each 95% interval resamples the 50 colour-swapped opening pairs 10,000 times, then transforms the 2.5th and 97.5th percentiles of the resulting match scores with the anchor held fixed. The intervals therefore quantify match sampling uncertainty, excluding uncertainty in the historical calibration.

At 100, 1,000, 10,000, and 100,000 searches, the harder opponent implies a rating 128, 102, 46, and 4 Elo higher than the easier opponent. The discrepancy narrows with search budget, reaching close agreement at the deepest point. Draw behaviour, calibration error, matchup effects, and sampling variation could contribute; the matches do not distinguish them. Selecting the score nearest 0.5 limits extrapolation from lopsided results.

C Supporting search and systems comparisons

The following controls provide the numerical comparisons supporting the search and systems decisions in Chapters 4 and 5. Each comparison retains its own workload and measurement scale.

C.1 Architecture shape and inference throughput

The width sweep used 12-block global-pooling CNNs with a dense policy head, BF16, batch 512, and one RTX 4070 SUPER. Each width was benchmarked in a separate process against an interleaved width-128 reference. Table C1 contrasts measured throughput with the inverse-width-squared estimate implied by convolutional arithmetic. Ratios below are normalized to the unrounded width-128 rate of 103,490 positions/s.

Table C.1: CNN width: measured throughput versus arithmetic prediction

Depth $\times$ width	Positions/s (k)	Measured ratio	Arithmetic ratio
12 $\times$ 96	133	1.284	1.778
12 $\times$ 112	95.9	0.926	1.306
12 $\times$ 120	93.4	0.903	1.138
12 $\times$ 128	103	1.000	1.000
12 $\times$ 136	59.7	0.577	0.886
12 $\times$ 144	63.8	0.617	0.790
12 $\times$ 152	53.8	0.520	0.709
12 $\times$ 160	61.8	0.597	0.640
12 $\times$ 176	52.4	0.507	0.529
12 $\times$ 192	41.5	0.401	0.444
12 $\times$ 224	35.5	0.343	0.327
12 $\times$ 256	31.1	0.301	0.250

The 112- and 120-channel models perform less arithmetic than the 128-channel model but serve fewer positions per second. The sharp loss at 136 channels likewise exceeds the arithmetic prediction. Width alone is therefore an unreliable proxy for inference cost.

Table C2 extends the comparison to depth and serving batch size. Ratios are relative to the specified reference at the same batch size. The width sweep is backed by per-process measurements; the depth/batch comparisons are transcribed benchmark summaries. Dashes indicate settings not measured.

Table C.2: Depth and width: throughput ratios across batch sizes

Model	Reference	Batch 512	Batch 320	Batch 64
20 $\times$ 128	14 $\times$ 152	1.36	1.20	0.730
10 $\times$ 176	14 $\times$ 152	1.35	1.24	1.34
13 $\times$ 160	14 $\times$ 152	1.23	1.13	1.10
14 $\times$ 160	14 $\times$ 152	–	1.05	1.01
15 $\times$ 160	14 $\times$ 152	–	0.980	0.930
16 $\times$ 160	14 $\times$ 152	–	0.918	0.960
14 $\times$ 176	14 $\times$ 152	–	0.888	0.956
11 $\times$ 224	18 $\times$ 176	1.10	1.24	1.66
34 $\times$ 128	18 $\times$ 176	1.06	1.16	0.556
19 $\times$ 176	18 $\times$ 176	–	0.948	0.970
20 $\times$ 176	18 $\times$ 176	–	0.899	0.912
22 $\times$ 160	18 $\times$ 176	–	0.995	0.838
4 $\times$ 224	12 $\times$ 128	0.978	1.09	2.36
6 $\times$ 176	12 $\times$ 128	0.977	1.01	1.74

The 20 $\times$ 128 network is faster than 14 $\times$ 152 at batch 512 but slower at batch 64. Such reversals motivate measuring candidate models at both self-play and interactive batch sizes rather than extrapolating from parameter count.

C.2 Replay-reuse controls

Approximately 90-minute controls at reuse ratios 4, 6.25, and 8 showed comparable strength at their shared evaluation boundaries despite different update rates. The retained ratio of four favours fresh self-play data. Its long-run effect is coupled to replay

capacity, optimization, and inference changes in the completed campaign. Reuse also changes the wall-clock pace of evaluation, publication, search-budget schedules, and replay growth, which advance at quantum boundaries. The configured credit per admitted row therefore describes the training schedule rather than the exact exposure of every distinct replay position.

C.3 Negative search and reuse controls

The learned pre-search allocator captured approximately 23% of the available improvement in deep-policy divergence at nearly matched search cost, but trailed non-adaptive online training by roughly 60–100 ladder Elo. The in-search stopper skipped approximately 14% of nominal simulations while improving generation cadence by only about 3% under actor-trainer overlap. Its paired strength differences, calculated as baseline minus stopper ($+1.7 \pm 9.9$ and -4.2 ± 10.1 Elo, standard errors), did not resolve an effect.

Exact graph search avoided only 0.0249% and 0.1769% of neural evaluations at 1,000 and 10,000 searches while being 8.63% and 8.28% slower. A bounded inference cache shared inside each self-play process had a 0.970% hit rate; disabling it made game updates about 0.88% faster and lowered peak worker memory about 7.12%. A wider unbounded repeat tracker observed at most about 4% reuse in the tested production-like workloads and itself cost throughput. In these workloads, the measured reuse was insufficient to offset the cost of detecting and exploiting it.

C.4 Throughput benchmarks

Host submission. The controlled 32-process self-play benchmark increased aggregate search throughput from 512,679 to 617,782 searches/s. On the same node restricted to 24 CPU cores, it increased from 215,265 to 496,036 searches/s. These are rates across the actor workload, including search and inference.

Neural inference. The matched runtime benchmark measured 75,889 positions/s with TensorRT FP16 and 40,716 with TorchScript BF16. These count positions evaluated by the network, not aggregate self-play searches.

Replay delivery. Compacting 5,000 producer shards into 25 containers increased loader throughput on 2.5 million rows from 3,943 to 32,579 samples/s. In a separate live interval, materialization appended 8,790 positions/s against an arrival rate of 1,365 accepted positions/s.

Training. An eight-GPU benchmark with global batch 2,048, bfloat16 autocast, and concurrent self-play processed 6,252 training samples/s. The actor-overlap sweep in Table 3 is a separate workload; its trainer and concurrent-search rates should be compared within that sweep.

Table C3 retains the unrounded trainer measurements. Chapter 5 presents the comparison alongside the scheduling decision. Complete-cycle time includes training and the remaining wait for self-play data.

Table C.3: Trainer throughput under actor overlap

Active actors	Training samples/s	Complete cycle (s)
0	25,275	–
8	21,492	117.1
16	17,139	112.9
32	9,210	111.2

The 1.86x TensorRT INT8 versus TorchScript search comparison used a production-shaped 400-visit actor workload, with different checkpoint weights, so the comparison includes both backend and model changes. The 23.3% replay-admission gain compared separate live 400- and 600-visit stages that also differed in checkpoint and actor scheduling. Absolute systems rates vary with CPU quota, GPU power, PCIe and NUMA layout, runtime versions, model shape, batching, and concurrent training.

D Reproducibility and release boundary

This appendix specifies the network representation, numerical recipe, and artifacts required to reproduce the software and evaluation. The released configuration supports new training runs; the published checkpoint fixes the model used for the reported results.

D.1 Network input and output contract

The network input is a $52 \times 8 \times 8$ tensor with channels listed in Table D1. "Own" denotes the player to move. For Black to move, ranks are reflected and piece colours are exchanged; files retain their order. Tensor row zero is the player's home rank, and column zero is file a. There is no separate absolute-colour plane. Spatial masks contain zeros and ones; flags and scalar values fill all 64 squares.

Table D.1: Chess input planes in tensor order (zero-based indices)

Plane indices	Feature, in channel order	Encoding
0, 1, 2, 3, 4, 5	Own pawn, knight, bishop, rook, queen, king	Piece-location masks
6, 7, 8, 9, 10, 11	Opponent pawn, knight, bishop, rook, queen, king	Piece-location masks
12	Own kingside castling right	Constant 0 or 1
13	Own queenside castling right	Constant 0 or 1
14	Opponent kingside castling right	Constant 0 or 1
15	Opponent queenside castling right	Constant 0 or 1
16	All own pieces	Occupancy mask
17	All opponent pieces	Occupancy mask
18	Pieces checking the player to move	Checker-location mask
19	En-passant target	One square, or all zeros
20	At least one earlier occurrence of this position	Constant 0 or 1
21	At least two earlier occurrences of this position	Constant 0 or 1
22, 23	Most recent move: origin, destination	Two single-square masks
24, 25	Second-most-recent move: origin, destination	Two single-square masks
26, 27	Third-most-recent move: origin, destination	Two single-square masks
28, 29	Fourth-most-recent move: origin, destination	Two single-square masks
30, 31	Fifth-most-recent move: origin, destination	Two single-square masks
32, 33	Sixth-most-recent move: origin, destination	Two single-square masks
34, 35	Seventh-most-recent move: origin, destination	Two single-square masks
36, 37	Eighth-most-recent move: origin, destination	Two single-square masks
38	Fixed checkerboard, with a1 set to one	Alternating 0/1 mask
39	Exactly one bishop per side, on opposite colours	Constant 0 or 1
40, 41, 42, 43, 44, 45	Pawn, knight, bishop, rook, queen, king balance	Own count minus opponent count
46	Halfmove clock	Integer count, capped at 100
47, 48, 49, 50, 51	Own pawn, knight, bishop, rook, queen counts	Integer counts

Channels 0–39 are binary; channels 40–51 contain unnormalized scalar values. Missing history entries are zero-filled. Move history stores origin and destination pairs for the last eight plies rather than complete board states; castling records the king’s actual destination. The checkerboard is fixed in canonical coordinates.

The shared backbone produces $160 \times 8 \times 8$ features. The policy head projects each square to a 128-dimensional token and forms query and key vectors whose scaled dot products score origin-destination pairs. These scores are gathered into 1,880 action logits: 1,792 ray or knight pairs and 88 explicit promotion actions. Promotion offsets distinguish queen, rook, bishop, and knight choices. Castling uses the king-to-own-rook pair in the action encoding, while en passant uses the pawn’s ordinary origin-destination pair. Illegal actions are masked before softmax over the legal moves.

The value branch uses a two-channel 1×1 convolution, batch normalization, ReLU, flattening, a 48-unit hidden layer, and three logits. Softmax produces win, draw, and loss probabilities for the player to move. The auxiliary branches produce a further 1,880 logits for the next searched policy and one scalar for remaining game length. They are used only in training; inference returns the primary policy logits and WDL probabilities.

D.2 Configuration and reported result

The fully expanded `chess-final-config.yaml` is the maintained entry point for training with the current recipe. Replication of the reported experiment instead uses its frozen source revision, resolved configuration, manifest, checkpoints, engines, and datasets. This separates future recipe updates from the fixed experimental record.

D.3 Expanded chess recipe settings

The final recipe ran on one node with eight RTX 4070 SUPER GPUs and 80 logical CPUs. Its network ladder contains 12×128 , 14×160 , and 19×176 residual CNNs with capped scaled post-activation branches and global-pooling context in every second block. The reported checkpoint uses the 14×160 stage.

The architecture uses a key-size-128 chess from-to policy head and a two-channel WDL head with a 48-unit hidden layer. The training-only next-searched-policy and remaining-game-length heads have loss weights 0.15 and 0.1. Primary policy and value losses each have weight 1.0. Terminal outcome targets are discounted by 0.998 per ply; the search-root-value blend rises from zero to 0.1 over its configured schedule, while search backup uses a separate 0.99 per-ply discount. At a capped game’s final position, a searched scalar value v is converted to a soft WDL target with $r = 1 - |v|$: win, draw, and loss receive $\max(v, 0) + r/3$, $r/3$, and $\max(-v, 0) + r/3$, respectively. Capped games omit the remaining-game-length loss. Batch-normalization folding, recalibration, ONNX export, and TensorRT refitting operate on a deployment copy, leaving the trainable model unfused.

Each 500-step training quantum uses eight ranks processing 256 positions each, for a global batch of 2,048. Nesterov SGD uses momentum 0.9, weight decay 0.0001, and gradient clipping at norm 1.0. The learning rate warms from zero to 0.1 over the first 1,000 optimizer steps and then follows the configured linear schedule to 0.01. Training uses bfloat16 and persistent trainer processes; `torch.compile` is disabled. QAT calibration uses 516 real evaluation positions and is refreshed at every publication boundary.

The 32 self-play actors run four per GPU, with 512 interleaved games per actor. Native inference uses batches of 320 with two outstanding batches per worker. Search uses exploration constant 1.5, reduced-parent FPU with reduction 0.2, forced playout coefficient 1.5, and Dirichlet epsilon 0.25 with alpha 0.3. Eight materializers convert completed games into the fixed-layout memory-mapped replay store; training credit is committed only against durable admitted rows. Restart positions require at least 15 plies remaining in the source game, absolute root value at most 0.8, and two or three leading actions covering 85% of visit mass. The played branch is marked used, and reservations keep workers from claiming the same alternative concurrently. Replay’s logical capacity grows through 0.6, 1.2, 2.0, 2.8, 4, 6, 8, 12, 16, and 20 million rows. Restart selection is 30% uniform and otherwise weighted by the square root of value correction, defined as half the absolute difference between searched root value and raw network value. Age and capacity bounds remove old states, and the archive is local to each worker.

The successor network trains on a captured replay snapshot for an average of 1.5 optimizer quanta per active-model quantum, with its own catch-up learning-rate clock. The configured match gate requires the successor to score at least 0.48 in two consecutive paired evaluations. The fully expanded config [10] specifies the stage-specific ladder-plateau thresholds and window.

D.4 Self-play schedules and promotion

Half the actors pause during each training quantum. The search schedule rises from 300 to 800 visits per move; the reported checkpoint lies in the 600-visit stage. Games start approximately equally often from random legal openings of up to eight plies and from eligible restart states, falling back to a random opening when no restart is available. Game caps increase from 150 to 250 plies, and greedy move selection starts later as training matures. Twenty percent of games are designated as no-resignation continuations to calibrate the resignation threshold against a 2.5% upper-bound target for mistakenly resigning a win or draw.

Each admitted replay position funds four training presentations. Sampling assigns 30% uniform probability and otherwise prioritizes bounded policy surprise. Candidate training begins when smoothed 64-search ladder gains fall below 15 Elo/hour for the small stage or 4 Elo/hour for the medium stage. Promotion uses the paired-match gate specified above. Training monitors run every 20 minutes using 50 paired openings, a three-rung adaptive Stockfish 13 bracket, and both 64-search and policy-only play.

D.5 Result identity and provenance

The experiment archive records the following provenance:

- Git revision and clean/dirty state, resolved YAML and SHA-256, and dependency-lock hash;
- operating image, software and driver versions, GPU, CPU, RAM, disk, and engine versions and archive hashes;
- evaluation dataset and opening-suite identities, hashes, raw matches, aggregates, commands, and interval method;
- run manifest, approval, coordinator logs, TensorBoard events, and resource telemetry;
- replay schema, capacity, occupancy, and reconciled volume counters;
- checkpoint and inference hashes, ONNX and TensorRT provenance, calibration positions, and fidelity reports;
- one digest covering the fetched archive or a checksummed artifact manifest.

The published INT8 ONNX has SHA-256 `d634abacae3c874eac6ded89f6af861eb81b509da638b5ad710587b1a08be658` at the immutable model-repository revision `dc8fccccb67ab5ec9e36267a165a9700b7dbf55f` [11]. The source release’s evidence index [10] records additional artifact hashes. The frozen training-source revision, resolved configuration, large run archives, and some exact evaluation inputs remain in the local result archive. The public recipe and model support inspection and a new run, but are not yet a self-contained package for exact-match reproduction of the reported experiment.

D.6 Reproducing the software

The public source release [10] contains setup and validation instructions. On production nodes, `deployment/setup_remote.sh` installs locked dependencies, builds the Release extension, installs pinned evaluation engines, and verifies engine execution. `deployment/run_control.sh` manages the run lifecycle.

Original project code and documentation, including this report, are available under the MIT License in the source release [10]. The published final model artifacts carry the same license in the model repository [11]. External dependencies, reference sources, and third-party data retain their own terms.

D.7 Reproducing evaluation

Matched evaluation uses the published checkpoint and inference artifact, paired opening suite, opponent binary, node limits, thread and hash settings, candidate search budget, parallelism, batching, and adjudication rules. Re-exporting with a different toolchain can change predictions and constitutes a new deployment comparison. [Appendix B](#) specifies the final match protocol and rating calculation.

Performance measurements distinguish isolated neural-network throughput, saturated multi-game search, and single-game latency. These measure different execution regimes and require separate benchmarks.

D.8 Reproducing plots and tables

Plots and tables are generated from archived JSON, CSV, TensorBoard, and manifest data, with extraction methods retained alongside the evidence. The underlying columns support recomputation of totals, rates, Elo estimates, and uncertainty intervals independently of the live dashboards.